TempusBench: An Evaluation Framework for Time-Series Forecasting

Denizalp Goktas*, Gerardo Riaño-Briceño*, Omkar Tekawade*, Alif Abdullah, Md Yasif Jamal, Vinesha Shaik, Arsh Singh, Aryan Nair, Chenkai Shen, Beatriz de Lucio, Alexandra Magnusson, Farhan Mashrur, Ahmed Abdulla, Shawrna Sen, Mahitha Thippireddy, Gregory Schwartz, Amy Greenwald

Simulacrum, New York City, NY, USA *Equal contribution.

Correspondence: deni@smlcrm.com, amy@smlcrm.com

October 6, 2026

arXiv:2604.11529v3 [cs.LG] 5 Oct 2026

ABSTRACT

Foundation models have transformed natural language processing and computer vision, and a rapidly growing literature on time-series foundation models (TSFMs) seeks to replicate this success in forecasting. While recent open-source models demonstrate the promise of TSFMs, the field lacks a comprehensive and community-accepted model evaluation framework. We see at least four major issues impeding progress on the development of such a framework. First, existing evaluation frameworks comprise benchmark forecasting tasks derived from often outdated datasets (e.g., M3), many of which lack clear metadata and overlap with the corpora used to pre-train TSFMs. Second, these frameworks evaluate models along a narrowly defined set of benchmark forecasting tasks, such as forecast horizon length or domain, but overlook core statistical properties such as non-stationarity and seasonality. Third, domain-specific models (e.g., XGBoost) are often compared unfairly, as existing frameworks do not enforce a systematic and consistent hyperparameter tuning convention for all models. Fourth, visualization tools for interpreting comparative performance are lacking. To address these issues, we introduce TempusBench, an open-source evaluation framework for TSFMs. TempusBench consists of 1) new datasets which are not included in existing TSFM pretraining corpora, 2) a set of novel benchmark tasks that go beyond existing ones, 3) a model evaluation pipeline with a standardized hyperparameter tuning protocol, and 4) a tensorboard-based visualization interface. We provide access to our code on GitHub: https://github.com/Smlcrm/TempusBench and maintain a live leaderboard at https://smlcrm.com/tempusbench.

1 Introduction

Foundation models are models trained on large and diverse datasets that can be used to solve a variety of downstream tasks. Their success in natural language processing (NLP) and computer vision has inspired an emerging literature on *time-series foundation models (TSFMs)*. (TSFMs) are models that take past time-series data (and possibly covariate time-series data) as input, and output future values (or distributions over them), typically formulated as neural networks trained via supervised learning. While about a dozen open-source TSFMs [1–14] are now available, comparing their performance to one another and to traditional domain-specific models (e.g., ARIMA [15], SVR [16, 17]) remains difficult. A

handful of evaluation frameworks [18–23] have been released, but the field still lacks comprehensive, community-accepted standards for model evaluation [24], which creates an impediment to replicating the success that foundation models have seen in NLP and computer vision [25].

In our view, existing evaluation frameworks face four major challenges. First, the evaluation ecosystem relies on outdated datasets such as M3 [26] and M4 [27], many lacking metadata (e.g., variable names). More importantly, the existing evaluation datasets overlap with the pretraining corpora of TSFMs, leading to inflated estimates of zero-shot generalization [28]. For example, except for Moirai2, all TSFMs assessed by GIFT-Eval include test data in their training corpora [23, 29]. Second, current frameworks define benchmark forecasting tasks only along narrow axes (i.e., forecast horizon, variate type, frequency, and domain). While useful, these dimensions neglect key statistical properties long studied in time-series analysis such as (non-)stationarity and seasonality. Since they do not evaluate such properties, existing frameworks cannot thoroughly evaluate model capabilities. Third, existing frameworks have not yet developed standardized hyperparameter tuning routines, leading to unfair comparisons between TSFMs and domain-specific models, as the performance of domain-specific models depends heavily on hyperparameter choices.[1] Indeed, as noted by practitioners [30], simple statistical models with well-chosen hyperparameters can outperform more complex ones, highlighting the need for consistent tuning routines. Fourth, evaluation typically reduces to numerical metrics such as mean squared error, which practitioners remark [31] provide limited interpretability. For instance, under GIFT-Eval, Seasonal Naive [26, 32] outperforms five open-source TSFMs, but this empirical observation offers no insight into the strength and weaknesses of TSFMs, since seasonal naive fails when seasonality is weak. Beyond quantitative scores, qualitative analyses—especially forecast visualizations—are essential.

To address these issues, we introduce TempusBench, an open-source evaluation framework. TempusBench consists of 1) new datasets which are not included in existing TSFM pretraining corpora, 2) a set of novel benchmark tasks that goes beyond existing ones, and 3) a model evaluation pipeline with a standardized hyperparameter tuning protocol, and 4) a tensorboard-based visualization interface.

Contributions. TempusBench includes 53 forecasting models,[2] a number of which (e.g., XGBoost) have not previously been considered by evaluation frameworks. Moreover, it is designed to overcome the aforementioned four issues by improving along the following dimensions. First, we introduce 70 new time-series datasets that are not based on existing time-series evaluation datasets, and that are not contained in the training corpus of open-source TSFMs released to date. Second, we propose new benchmark task types that extend beyond horizon length, variate type, frequency, and domain. These include categories based on stationarity, seasonality, variable type (continuous, count, binary, categorical), dataset size (small vs. large), sparsity (sparse vs. dense), and quality (noisy vs. measurement error). Third, we introduce a model evaluation pipeline that runs a standardized and automated hyperparameter search procedure for all forecasting models with hyperparameters, facilitating a fair comparison of all forecasting methods. Fourth, TempusBench comes packaged with a tensorboard-based visualization application that easily allows researchers and practitioners to visualize and interpret the performance of different models on different tasks.

[1] TSFMs require hyperparameter searches during pretraining, but not during evaluation.

[2] In Section C, we list the families of forecasters in TempusBench (Table 9) and describe many of them. 47 of these models are multivariate forecasters with covariate support, while 50 are multivariate forecasters without covariate support.

Table 1. Property comparisons of various forecasting benchmarks.

Property	Monash [18]	TFB [19]	LTSF [20]	BasicTS+ [21]	ProbTS [22]	GIFT-Eval [23]	TempusBench
Frequency Range	Second to Year	Minute to Year	Minute to Week	Minute to Day	Minute to Week	Second to Year	Second to Year
Num. Domains	7	6	5	3	5	7	10
Train/Test data leak	✓	✓	✓	✓	✓	✓	✗
Variate Types	Uni	Uni/Multi	Multi	Multi	Multi	Uni/Multi	Uni/Multi
Prediction Length	Short	Short	Long	Short/Long	Short/Long	Short/Long	Short/Long
Stat. Benchmarks	✗	✗	✗	✗	✗	✗	✓
Forecaster types	Stat./DL	Stat./DL	Stat./DL	Stat./DL	Stat./DL/FM	Stat./DL/FM	Stat./ML/DL/FM
Hyperparam. autotuning	✗	✗	✗	✗	✗	✗	✓

2 Background

We refer the reader to Section A for the notational convention we adopt, and for additional mathematical preliminaries and evaluation metric definitions.

Forecasting Tasks. A *(time-series) forecasting task* $\mathcal{T} \doteq (l, h, n, m, (\mathcal{X}_j)_{j\in[n]}, (\mathcal{Y}_i)_{i\in[m]}, \boldsymbol{X}, \boldsymbol{Y})$ comprises a *context length* $l \in \mathbb{N}$, a *forecast horizon* $h \in \mathbb{N}$, $m \in \mathbb{N}$ *target time-series* $\boldsymbol{Y} = (\boldsymbol{y}_1, \dots, \boldsymbol{y}_m)^T$, where for each variate $i \in [m]$, entries of $\boldsymbol{y}_i \in \mathcal{Y}_i^l$ take values y_{it}, for $t \in [l]$, from a *set of target values* $\mathcal{Y}_i \subseteq \mathbb{R}$, and $n \in \mathbb{N}$ *covariate time-series* $\boldsymbol{X} = (\boldsymbol{x}_1, \dots, \boldsymbol{x}_n)^T$, where for each covariate $j \in [n]$, $\boldsymbol{x}_j \in \mathcal{X}_j^{l+h}$ takes values x_{jt}, for $t \in [l+h]$, from a *set of covariate values* $\mathcal{X}_j \subseteq \mathbb{R}$. We denote the *joint set of covariate values* by $\mathcal{X}^{l+h} \doteq \bigtimes_{j\in[n]} \mathcal{X}_j^{l+h}$ and the *joint set of target variate values* by $\mathcal{Y}^l \doteq \bigtimes_{i\in[m]} \mathcal{Y}_i^l$.

A forecasting task $\mathcal{T}$ is said to be *univariate* iff $m = 1$; if $m > 1$, it is said to be *multivariate*. A *covariate* forecasting task $\mathcal{T}$ is one for which $n > 0$. A forecasting task $\mathcal{T}$ is said to be continuous (resp. count | categorical | binary) iff for all $i \in [m]$ $\mathcal{Y}_i \subseteq \mathbb{R}$ is a continuous set (resp. $\mathcal{Y}_i = \mathbb{N} \mid \mathcal{Y}_i \subsetneq \mathbb{N} \mid \mathcal{Y}_i = \{0, 1\}$).

Forecasters. Given a forecasting task $\mathcal{T}$, a *(point) forecast* is a matrix $\widehat{\boldsymbol{Y}} \doteq (\widehat{\boldsymbol{y}}_1, \dots, \widehat{\boldsymbol{y}}_m)^T$ s.t. for all target variates $i \in [m]$, $\widehat{\boldsymbol{y}}_i \in \mathcal{Y}_i^h$ represents forecasted values of variate i for h time steps. A *(point) forecasting model* is a mapping $\boldsymbol{F} : \mathcal{X}^{l+h} \times \mathcal{Y}^l \to \mathcal{Y}^h$ whose output is a forecast for $\mathcal{T}$: i.e., $\boldsymbol{F}(\boldsymbol{X}, \boldsymbol{Y}) \doteq (\boldsymbol{f}_1(\boldsymbol{X}, \boldsymbol{Y}), \dots, \boldsymbol{f}_m(\boldsymbol{X}, \boldsymbol{Y}))^T = \widehat{\boldsymbol{Y}}$. More generally, a *probabilistic forecasting model* is a mapping $\pi : \mathcal{X}^{l+h} \times \mathcal{Y}^l \to \Delta(\mathcal{Y}^h)$ s.t. $\pi(\boldsymbol{X}, \boldsymbol{Y})[\widehat{\boldsymbol{Y}}]$ denotes the realization probability of $\widehat{\boldsymbol{Y}} \in \mathcal{Y}^h$.

Forecasting Evaluation Frameworks. In practice, many forecasters $\boldsymbol{F}^\theta : \mathcal{X}^{l+h} \times \mathcal{Y}^l \to \mathcal{Y}^h$ depend on hyperparameters $\theta \in \Theta$, so that it is often more appropriate to consider a *family of forecasters* $\mathcal{F}^\Theta \doteq \{\boldsymbol{F}^\theta\}_{\theta\in\Theta}$, and then choose the forecaster with parameters best suited to the forecasting task at hand.[3] A forecaster evaluation framework $\mathcal{B} \doteq (p, q, \mathcal{E}, \{\Theta_i\}_{i=1}^p, \{\mathcal{F}^{\Theta_i}\}_{i=1}^p, \{\mathcal{T}_j\}_{j\in[q]})$ comprises a family of $\{\mathcal{F}^{\Theta_i}\}_{i\in[p]}$ of $p \in \mathbb{N}$ forecasters, each one defined by a set of hyperparameters Θ_i; $q \in \mathbb{N}$ forecasting tasks (or, colloquially, *benchmark tasks* or just *benchmarks*) $\{\mathcal{T}_j\}_{j\in[q]}$; and a hyperparameter tuner $\mathcal{E}$, which takes as input a benchmark and suitable hyperparameters.

[3] For instance, the forecast of an ARIMA model is dependent on choices of hyperparameters given by the order of the number of time lags, the degree of differencing, and the order of the moving-average model. It thus feels more appropriate to talk about the family of ARIMA models.

Table 2. Taxonomy of all univariate, multivariate, and covariate tasks included in TempusBench.

Category	Benchmark Tasks
Movement	Stationary, Non-Stationary
Data Quality	Noisy data, Data with measurement error
Frequency	Seconds, Minutes, Hours, Days, Weeks, Months, Quarterly, Years
Context Length	12, 18, 32, 45, 64, 128, 200, 256, 450, 512, 1024, 1536, 2048
Forecast Horizon	4, 5, 6, 8, 16, 17, 22, 30, 33, 35, 39, 42, 55, 64
Seasonality	Cyclical, Non-Stationary cyclical, Regressive, Irregular, Additive, Multiplicative
Domain	Energy, Transport, Climate, Software, Web, Sales, Nature, Econ., Healthcare, Manufacturing
Dataset Coverage	sparse, dense
Target Type	continuous, count, binary, categorical

3 TempusBench

TempusBench, denoted $\mathcal{B}^{\text{TB}}$, is a forecasting evaluation framework with a hyperparameter tuner $\mathcal{E}^{\text{TB}}$ that optimizes using the following time-series cross validation procedure: given a benchmark task, a set of hyperparameters, and a family of forecasters, 1) rolling evaluation windows are generated by sliding a fixed-size window along the target time series, where each window is partitioned into context, training, and validation segments, and the stride is set equal to the length of the validation segment; 2) for all hyperparameter combinations, all models are fit, given available past data (context + training data), and evaluated on the corresponding validation segments; the hyperparameters selected are those corresponding to the lowest validation loss;[4] 3) the test loss for each metric (e.g., MSE, RMSE, etc.) is then computed using the hyperparameters selected, and these losses are averaged across all windows to obtain the final metric values. This rolling window approach avoids the look-ahead bias that would arise from standard k-fold cross validation, and instead respects temporal dependencies in time-series data by maintaining strict temporal ordering.

In Table 2, we present a taxonomy of the benchmarks included in TempusBench. We summarize the actual benchmarks in Tables 11, 12, and 13 (Section D).

4 Result Aggregation Procedure

After evaluating multiple forecasting models across a diverse set of benchmark tasks, we require aggregation methods to summarize and compare model performance at the aggregate level. This section describes two complementary aggregation procedures: *average win rate* and *skill score*.

In practice, some models may not produce valid results on certain tasks (e.g., due to computational failures or data incompatibilities), resulting in missing values. Our aggregation procedures handle these missing values gracefully by excluding unavailable comparisons.

Problem Setup. Let p denote the number of models under evaluation and let q denote the number of benchmark tasks. We compute an evaluation (i.e., error) metric $\ell^{i,j}$ (e.g., MAE, RMSE, etc.) for all models $i \in [p]$ on all tasks $j \in [q]$, and then aggregate those errors in two ways.

Average Win Rate. The *average win rate* W_i for model i quantifies the probability that model i achieves lower error than another randomly chosen model $i' \neq i$ on a randomly chosen benchmark task. This

[4] In TempusBench, we use MAE as the validation loss function for optimizing hyperparameters.

metric provides a pairwise comparison perspective that is robust to the absolute scale of errors across different tasks.

Formally, for model i, the average win rate is computed as:

$$W_i = \frac{1}{|\mathcal{C}_i|} \sum_{j \in [q]} \sum_{\substack{i' \in [p] \\ i' \neq i}} w_{i,i',j} \tag{1}$$

The win indicator $w_{i,i',j}$ is defined as follows:

$$w_{i,i',j} = \begin{cases} 1 & \text{if } \ell^{i,j} < \ell^{i',j} \text{ and both values are valid} \\ 0.5 & \text{if } \ell^{i,j} = \ell^{i',j} \text{ and both values are valid} \\ 0 & \text{if } \ell^{i,j} > \ell^{i',j} \text{ and both values are valid} \\ 0 & \text{if either value is missing.} \end{cases} \tag{2}$$

The normalization factor $|\mathcal{C}_i|$ accounts for the actual number of valid comparisons:

$$\mathcal{C}_i = \{j \in [q], i' \in [p], i' \neq i \mid \ell^{i,j} \text{ and } \ell^{i',j} \text{ are both valid}\} \tag{3}$$

Skill Score. The *skill score* S_i for model i quantifies how much the model reduces forecasting error compared to a fixed baseline model $\beta \neq i$. Unlike win rate, which compares models in a pairwise manner, skill score provides an absolute measure of improvement relative to a reference model (typically a simple baseline, such as Seasonal Naive Forecasting).

For model i relative to baseline β, the skill score is computed as:

$$S_i = 1 - \left(\prod_{j \in \mathcal{R}_i} \text{clip}\left(\frac{\ell^{i,j}}{\ell^{\beta,j}}; \ell, u \right) \right)^{1/|\mathcal{R}_i|}, \tag{4}$$

where $\mathcal{R}_i = \{j \in [q] : \ell^{i,j} \text{ and } \ell^{\beta,j} \text{ are both valid}\}$ is the set of tasks for which both model i and baseline β have valid results, and $\text{clip}(x; \ell, u) = \max(\ell, \min(x, u))$ clips the relative error ratio to the interval $[\ell, u]$ with $\ell = 10^{-2}$ and $u = 100$.

The clipping operation prevents extreme relative errors (e.g., division by near-zero baseline errors) from dominating the geometric mean. We handle the edge case $\ell^{\beta,j} = 0$ as follows:

$$\frac{\ell^{i,j}}{\ell^{\beta,j}} = \begin{cases} 1 & \text{if } \ell^{i,j} = 0, \\ u & \text{if } \ell^{i,j} > 0. \end{cases} \tag{5}$$

The skill score interpretation is straightforward:

- $S_i > 0$: Model i performs better than the baseline (lower relative error).
- $S_i = 0$: Model i performs equivalently to the baseline.
- $S_i < 0$: Model i performs worse than the baseline.

Geometric Mean Rationale. The skill score uses a geometric mean (via the product raised to the reciprocal power) rather than an arithmetic mean for aggregating relative errors across tasks. This choice

has several advantages: 1. *Scale invariance*: The geometric mean is invariant to multiplicative scaling, so that high-error tasks do not dominate. Tasks with different error magnitudes contribute proportionally. 2. *Symmetry*: The geometric mean treats improvements and degradations symmetrically (e.g., a 2× improvement and a 2× degradation cancel out in the geometric mean). 3. *Robustness*: The geometric mean is less sensitive to outliers than the arithmetic mean, which is important when aggregating across diverse benchmark tasks.

Implementation Details. Both aggregation procedures are available in TempusBench, which provides two aggregator classes: `WinRate` and `SkillScore`, both inheriting from `BaseAggregator`. Each can be instantiated with a pivot table, where rows represent models $i \in [p]$, columns represent benchmark tasks $j \in [q]$, and cell (i, j) stores error $\ell^{i,j}$; and, in the case of `SkillScore`, a baseline model β (default: $\beta =$ `seasonal_naive`). Missing values are automatically detected and excluded from the aggregation, ensuring that models are only compared on tasks where both models have valid results.

4.1 Average Win Rate Results

In Tables 3, 4, and 5, we report the per-metric average win rates for deterministic and probabilistic models, across univariate, multivariate, and covariate forecasting tasks, respectively. All win rates compare a model's performance pairwise against competing models on the evaluated tasks. Higher win rates indicate that a model consistently outperforms its pairwise competitors, while lower win rates indicate more frequent underperformance.

Univariate (see Table 3). The strongest models (TiRex, TiRex 1.1, TimesFM 2.5, Moirai 2.0, and Chronos-2) achieve roughly 64%–75% average win rates across both stochastic (CRPS, QS, WIS) and deterministic (MAE, MAPE, MASE, RMSE) metrics—indicating consistent superiority across task pairings. TiRex leads across all measured metrics, peaking at over 75% win rates for MAE and MASE, closely followed by its 1.1 variant and TimesFM 2.5. Chronos-2 and Moirai 2.0 also show robust performance, winning in about two-thirds of the evaluated tasks. Lighter models, such as the Chronos-Bolt series and PatchTST-FM, remain highly competitive with win rates generally between 55% and 66%, illustrating solid baseline capabilities even at smaller parameter counts.

Multivariate (see Table 4). TiRex and TiRex 1.1 establish a dominant lead on multivariate tasks, attaining win rates between 78% on stochastic metrics and over 85% on deterministic metrics (MAE, MASE, RMSE). TimesFM 2.5 and Chronos-2 maintain strong second-tier performance, winning over 73% of their respective evaluations. Notably, Chronos-2 exhibits heterogeneous metric behavior: high win rates on CRPS, QS, WIS, MAE, MASE, and RMSE (77%–83%), but a noticeable drop on MAPE (64.5%). Models like PatchTST-FM and TabPFN-TS show highly competitive deterministic scores (particularly MAE and MASE around 72%–76%) despite lower comparative stochastic performance.

Covariate (see Table 5). Chronos-2 and TabPFN-TS demonstrate the strongest performance in covariate forecasting. Chronos-2 dominates the probabilistic metrics with an 83% win rate across CRPS, QS, and WIS, while TabPFN-TS commands the deterministic metrics, peaking at 82% on MAE and MASE, and 80.4% on MAPE. Chronos-2-Small similarly shows robust performance, outperforming many significantly larger models. Notably, TiRex and TiRex 1.1, which dominated univariate and multivariate tasks, drop to the 10th and 11th ranks here, with win rates hovering around 60%–67%. Several classical and statistical baselines, such as Prophet, perform surprisingly well on deterministic metrics (e.g., nearly 70% win rate on RMSE).

Table 3. Univariate tasks – Average win rates by metric for deterministic & probabilistic models

Rank	Model	Params (M)	Stochastic			Deterministic			
			CRPS	QS	WIS	MAE	MAPE	MASE	RMSE
1	TiRex (NX-AI)	300	**70.4%**	**70.4%**	**70.4%**	**75.1%**	**70.5%**	**75.1%**	**73.4%**
2	TiRex 1.1 (NX-AI)	300	68.5%	68.5%	68.5%	74.3%	69.7%	74.3%	71.5%
3	TimesFM 2.5 (Google)	200	66.4%	66.4%	66.4%	73.3%	69.4%	73.4%	71.6%
4	Moirai 2.0 (Salesforce)	12	64.0%	64.0%	64.0%	69.6%	67.1%	69.6%	68.8%
5	Chronos-2 (Amazon)	120	64.6%	64.6%	64.6%	66.8%	63.0%	66.8%	68.5%
6	TimesFM 500M (Google)	500	58.7%	58.7%	58.7%	68.5%	65.9%	68.5%	66.7%
7	PatchTST-FM (IBM)	260	62.8%	62.8%	62.8%	63.8%	62.1%	63.9%	65.0%
8	Chronos-Bolt-Mini (Amazon)	21	56.1%	56.1%	56.1%	66.5%	65.2%	66.6%	66.3%
9	TimesFM 200M (Google)	200	—	—	—	63.6%	63.0%	63.6%	62.1%
10	Chronos-Bolt-Tiny (Amazon)	9	56.7%	56.7%	56.7%	66.0%	65.0%	66.1%	65.7%
11	Chronos-2-Small (Amazon)	40	60.3%	60.3%	60.3%	63.4%	61.1%	63.5%	65.0%
12	Chronos-Bolt-Base (Amazon)	205	55.5%	55.5%	55.5%	64.5%	64.7%	64.6%	63.9%
13	Chronos-Bolt-Small (Amazon)	48	52.7%	52.7%	52.7%	63.7%	62.4%	63.8%	63.3%
14	Moirai-Base (Salesforce)	91	60.1%	60.1%	60.1%	58.5%	56.2%	58.5%	60.4%
15	TabPFN-TS (Prior Labs)	11	50.3%	50.3%	50.3%	62.6%	59.9%	62.6%	61.7%
16	Chronos-Large (Amazon)	710	45.2%	45.2%	45.2%	62.6%	63.7%	62.7%	61.6%
17	Moirai-Large (Salesforce)	311	56.6%	56.6%	56.6%	57.7%	55.1%	57.8%	58.7%
18	Chronos-Mini (Amazon)	20	45.4%	45.4%	45.4%	62.6%	62.0%	62.5%	60.7%
19	Granite FlowState (IBM)	9.1	48.9%	48.9%	48.9%	58.2%	59.0%	58.2%	58.1%
20	Moirai-Small (Salesforce)	14	54.4%	54.4%	54.4%	56.2%	52.9%	56.2%	57.5%
21	Chronos-Tiny (Amazon)	8	44.4%	44.4%	44.4%	59.1%	61.1%	59.0%	60.0%
22	N-HiTS (Nixtla)	—	—	—	—	56.2%	55.3%	55.4%	51.9%
23	Kairos-50M (Foundation Model Research)	50	45.8%	45.8%	45.8%	57.9%	56.4%	57.9%	56.8%
24	Chronos-Small (Amazon)	46	41.6%	41.6%	41.6%	58.0%	57.7%	58.0%	59.2%
25	N-BEATS (Nixtla)	—	—	—	—	53.1%	53.9%	53.1%	52.3%
26	TOTO (Datadog)	151	63.2%	63.2%	63.2%	50.3%	51.3%	50.4%	40.3%
27	Kairos-10M (Foundation Model Research)	10	42.0%	42.0%	42.0%	55.2%	55.7%	55.3%	55.6%
28	Chronos-Base (Amazon)	200	38.5%	38.5%	38.5%	56.1%	58.0%	56.1%	55.1%
29	LSTM (Nixtla)	—	—	—	—	50.4%	49.7%	50.2%	47.2%
30	Moirai-MoE (Salesforce)	86	49.5%	49.5%	49.5%	49.6%	48.9%	49.7%	48.4%
31	Kairos-23M (Foundation Model Research)	23	40.1%	40.1%	40.1%	51.2%	49.7%	51.2%	53.0%
32	PatchTSMixer (IBM)	—	—	—	—	43.4%	43.9%	43.6%	44.7%
33	iTransformer (THUML)	—	—	—	—	44.2%	43.1%	44.0%	42.7%
34	SVR (scikit-learn)	—	—	—	—	43.8%	41.9%	43.8%	44.3%
35	Random Forest (scikit-learn)	—	—	—	—	43.3%	42.6%	43.2%	43.5%
36	ARIMA (Statsmodels)	—	—	—	—	43.2%	41.8%	42.1%	41.0%
37	Exponential Smoothing (Statsmodels)	—	—	—	—	42.1%	40.1%	42.1%	40.5%
38	Croston (Statsmodels)	—	—	—	—	39.4%	38.9%	40.3%	38.0%
39	TTM R2 (IBM)	1.7	—	—	—	36.2%	41.4%	36.1%	39.3%
40	TTM R1 (IBM)	1	—	—	—	36.0%	41.2%	35.9%	39.2%
41	Time-MoE-200M (Shanghai AI Lab)	200	—	—	—	35.5%	41.0%	35.5%	38.8%
42	Time-MoE-50M (Shanghai AI Lab)	50	—	—	—	35.5%	41.0%	35.5%	38.8%
43	TTM R2.1 (IBM)	1.7	—	—	—	35.4%	40.3%	35.2%	38.8%
44	MOMENT-Large (CMU)	385	—	—	—	35.5%	39.4%	35.3%	37.9%
45	XGBoost (DMLC)	—	—	—	—	35.5%	36.2%	36.5%	37.0%
46	Prophet (Meta)	—	—	—	—	35.4%	36.8%	35.4%	37.3%
47	MOMENT-Base (CMU)	125	—	—	—	33.6%	36.6%	33.3%	35.5%
48	MOMENT-Small (CMU)	40	—	—	—	31.3%	34.8%	31.2%	32.9%
49	Seasonal Naive (Sktime)	—	—	—	—	29.4%	29.8%	29.7%	27.6%
50	Lag-Llama (Nixtla)	10	18.6%	18.6%	18.6%	26.0%	29.3%	26.1%	25.1%
51	PatchTST-Granite (IBM)	40	15.4%	15.4%	15.4%	25.7%	27.6%	25.6%	30.3%
52	Theta (Sktime)	—	—	—	—	21.7%	23.6%	21.9%	23.2%
53	Sundial (THUML)	128	3.3%	3.3%	3.3%	3.1%	3.2%	3.1%	3.0%

Table 4. Multivariate tasks – Average win rates by metric for deterministic & probabilistic models

Rank	Model	Params (M)	Stochastic			Deterministic			
			CRPS	QS	WIS	MAE	MAPE	MASE	RMSE
1	TiRex (NX-AI)	300	78.1%	78.1%	78.1%	**86.2%**	**80.6%**	**86.2%**	**85.9%**
2	TiRex 1.1 (NX-AI)	300	78.3%	78.3%	78.3%	85.3%	78.9%	85.3%	84.8%
3	TimesFM 2.5 (Google)	200	73.3%	73.3%	73.3%	83.6%	76.7%	83.5%	82.5%
4	Chronos-2 (Amazon)	120	77.8%	77.8%	77.8%	81.3%	64.5%	81.1%	83.6%
5	Chronos-2-Small (Amazon)	40	74.8%	74.8%	74.8%	75.9%	66.7%	75.7%	81.0%
6	TimesFM 500M (Google)	500	66.0%	66.0%	66.0%	79.6%	67.1%	79.6%	77.8%
7	PatchTST-FM (IBM)	260	73.3%	73.3%	73.3%	76.0%	60.2%	75.9%	76.8%
8	TabPFN-TS (Prior Labs)	11	55.9%	55.9%	55.9%	72.6%	67.1%	72.3%	72.2%
9	Chronos-Bolt-Mini (Amazon)	21	53.6%	53.6%	53.6%	74.5%	65.8%	74.3%	72.1%
10	TimesFM 200M (Google)	200	—	—	—	68.9%	60.4%	69.0%	66.1%
11	Chronos-Bolt-Small (Amazon)	48	55.7%	55.7%	55.7%	73.6%	57.4%	73.5%	75.0%
12	Chronos-Bolt-Base (Amazon)	205	53.2%	53.2%	53.2%	72.2%	64.8%	72.1%	72.5%
13	Chronos-Bolt-Tiny (Amazon)	9	47.8%	47.8%	47.8%	70.0%	59.8%	69.9%	71.7%
14	Moirai 2.0 (Salesforce)	12	51.2%	51.2%	51.2%	65.3%	56.1%	65.0%	66.0%
15	Kairos-50M (Foundation Model Research)	50	48.1%	48.1%	48.1%	60.5%	67.5%	60.4%	57.8%
16	Kairos-23M (Foundation Model Research)	23	43.3%	43.3%	43.3%	62.6%	58.2%	62.6%	57.8%
17	PatchTSMixer (IBM)	—	—	—	—	52.9%	50.2%	52.4%	56.6%
18	Granite FlowState (IBM)	9.1	41.2%	41.2%	41.2%	56.6%	60.4%	56.2%	58.3%
19	N-HiTS (Nixtla)	—	—	—	—	55.4%	36.9%	54.6%	55.3%
20	Kairos-10M (Foundation Model Research)	10	35.1%	35.1%	35.1%	54.0%	61.5%	54.0%	56.5%
21	Chronos-Mini (Amazon)	20	35.3%	35.3%	35.3%	58.1%	46.8%	57.8%	56.2%
22	Chronos-Base (Amazon)	200	35.7%	35.7%	35.7%	58.9%	46.8%	58.8%	52.1%
23	N-BEATS (Nixtla)	—	—	—	—	51.9%	32.5%	52.8%	53.9%
24	Chronos-Small (Amazon)	46	34.1%	34.1%	34.1%	56.2%	50.4%	56.1%	50.4%
25	Chronos-Tiny (Amazon)	8	32.9%	32.9%	32.9%	54.1%	46.2%	54.2%	52.6%
26	Chronos-Large (Amazon)	710	32.9%	32.9%	32.9%	52.9%	44.8%	52.7%	48.9%
27	TOTO (Datadog)	151	**78.7%**	**78.7%**	**78.7%**	34.4%	34.8%	34.5%	19.5%
28	iTransformer (THUML)	—	—	—	—	45.5%	38.4%	45.2%	45.1%
29	Moirai-Large (Salesforce)	311	56.4%	56.4%	56.4%	39.8%	33.6%	39.7%	38.9%
30	SVR (scikit-learn)	—	—	—	—	42.1%	44.9%	42.1%	44.0%
31	Prophet (Meta)	—	—	—	—	38.9%	47.2%	38.5%	44.6%
32	TTM R2 (IBM)	1.7	—	—	—	36.0%	56.6%	35.8%	37.8%
33	TTM R2.1 (IBM)	1.7	—	—	—	35.7%	55.4%	35.5%	38.0%
34	Croston (Statsmodels)	—	—	—	—	39.1%	45.7%	39.7%	39.2%
35	TTM R1 (IBM)	1	—	—	—	35.3%	55.7%	35.1%	37.2%
36	Moirai-Base (Salesforce)	91	49.1%	49.1%	49.1%	37.4%	33.0%	37.2%	34.2%
37	MOMENT-Large (CMU)	385	—	—	—	35.5%	49.1%	35.2%	36.6%
38	ARIMA (Statsmodels)	—	—	—	—	33.2%	48.0%	34.4%	37.8%
39	Random Forest (scikit-learn)	—	—	—	—	35.6%	41.1%	35.4%	38.5%
40	Time-MoE-200M (Shanghai AI Lab)	200	—	—	—	29.2%	52.5%	29.1%	33.7%
41	Time-MoE-50M (Shanghai AI Lab)	50	—	—	—	29.2%	52.5%	29.1%	33.7%
42	MOMENT-Base (CMU)	125	—	—	—	31.8%	45.9%	31.6%	32.3%
43	LSTM (Nixtla)	—	—	—	—	32.8%	38.7%	35.8%	30.7%
44	MOMENT-Small (CMU)	40	—	—	—	30.4%	42.4%	30.2%	30.5%
45	Moirai-Small (Salesforce)	14	31.5%	31.5%	31.5%	33.2%	30.6%	33.0%	32.4%
46	Exponential Smoothing (Statsmodels)	—	—	—	—	27.2%	35.4%	29.7%	30.0%
47	Seasonal Naive (Sktime)	—	—	—	—	26.7%	39.5%	25.4%	24.1%
48	PatchTST-Granite (IBM)	40	6.6%	6.7%	6.7%	18.0%	22.6%	17.8%	23.4%
49	Theta (Sktime)	—	—	—	—	13.5%	24.2%	13.5%	12.7%
50	Sundial (THUML)	128	0.0%	0.0%	0.0%	0.2%	3.9%	0.2%	0.6%

Table 5. Covariate tasks – Average win rates by metric for deterministic & probabilistic models

Rank	Model	Params (M)	Stochastic			Deterministic			
			CRPS	QS	WIS	MAE	MAPE	MASE	RMSE
1	Chronos-2 (Amazon)	120	**83.0%**	**83.0%**	**83.0%**	79.1%	72.1%	79.3%	**80.7%**
2	TabPFN-TS (Prior Labs)	11	77.0%	77.0%	77.0%	**82.0%**	**80.4%**	**82.2%**	72.3%
3	Chronos-2-Small (Amazon)	40	80.3%	80.3%	80.3%	74.1%	62.3%	74.2%	79.7%
4	TimesFM 200M (Google)	200	—	—	—	72.2%	71.0%	72.1%	65.4%
5	Chronos-Bolt-Base (Amazon)	205	61.1%	61.1%	61.1%	67.2%	69.3%	67.2%	67.9%
6	Chronos-Bolt-Tiny (Amazon)	9	54.8%	54.8%	54.8%	69.6%	71.1%	69.7%	70.0%
7	Prophet (Meta)	—	—	—	—	64.3%	62.5%	64.6%	69.9%
8	Chronos-Bolt-Small (Amazon)	48	57.7%	57.7%	57.7%	68.0%	67.9%	68.0%	69.4%
9	Chronos-Bolt-Mini (Amazon)	21	58.0%	58.0%	58.0%	67.1%	68.3%	67.2%	68.8%
10	TiRex (NX-AI)	300	62.4%	62.4%	62.4%	67.3%	63.6%	67.5%	60.8%
11	TiRex 1.1 (NX-AI)	300	60.5%	60.5%	60.5%	67.5%	63.2%	67.8%	61.5%
12	TimesFM 2.5 (Google)	200	55.6%	55.6%	55.6%	65.3%	61.7%	65.4%	62.4%
13	PatchTST-FM (IBM)	260	59.1%	59.1%	59.1%	58.4%	61.5%	58.4%	65.9%
14	Moirai 2.0 (Salesforce)	12	54.5%	54.5%	54.5%	65.7%	62.0%	65.8%	57.5%
15	Kairos-50M (Foundation Model Research)	50	55.0%	55.0%	55.0%	62.3%	61.1%	62.4%	59.0%
16	ARIMA (Statsmodels)	—	—	—	—	58.9%	56.1%	57.0%	62.3%
17	Kairos-10M (Foundation Model Research)	10	49.0%	49.0%	49.0%	60.6%	58.4%	60.6%	57.0%
18	TimesFM 500M (Google)	500	52.1%	52.1%	52.1%	58.2%	57.7%	58.3%	55.0%
19	Moirai-Small (Salesforce)	14	54.6%	54.6%	54.6%	51.4%	53.4%	51.5%	55.9%
20	Kairos-23M (Foundation Model Research)	23	43.4%	43.4%	43.4%	57.5%	55.3%	57.6%	54.9%
21	Chronos-Mini (Amazon)	20	34.1%	34.1%	34.1%	61.1%	62.5%	61.0%	53.6%
22	PatchTSMixer (IBM)	—	—	—	—	49.2%	52.7%	48.1%	56.3%
23	Granite FlowState (IBM)	9.1	42.3%	42.3%	42.3%	56.2%	54.1%	56.3%	52.9%
24	TOTO (Datadog)	151	57.5%	57.5%	57.5%	45.6%	47.1%	45.8%	41.5%
25	Chronos-Tiny (Amazon)	8	31.2%	31.2%	31.2%	57.7%	55.8%	57.7%	49.7%
26	Chronos-Base (Amazon)	200	33.2%	33.2%	33.2%	53.7%	54.2%	53.7%	55.4%
27	Exponential Smoothing (Statsmodels)	—	—	—	—	45.2%	47.7%	45.6%	53.1%
28	Chronos-Small (Amazon)	46	26.9%	26.9%	26.9%	55.8%	55.3%	56.0%	51.6%
29	Moirai-Base (Salesforce)	91	48.7%	48.7%	48.7%	44.6%	44.3%	45.0%	48.0%
30	LSTM (Nixtla)	—	—	—	—	48.8%	49.0%	47.3%	33.9%
31	Croston (Statsmodels)	—	—	—	—	44.0%	42.7%	45.6%	46.3%
32	TTM R2 (IBM)	1.7	—	—	—	39.5%	47.2%	39.5%	45.4%
33	TTM R1 (IBM)	1	—	—	—	38.6%	47.1%	38.7%	45.2%
34	MOMENT-Large (CMU)	385	—	—	—	38.3%	44.9%	38.1%	44.3%
35	iTransformer (THUML)	—	—	—	—	38.4%	36.6%	38.5%	41.6%
36	Random Forest (scikit-learn)	—	—	—	—	34.1%	39.3%	34.2%	39.7%
37	Time-MoE-200M (Shanghai AI Lab)	200	—	—	—	33.2%	40.3%	33.3%	38.6%
38	Time-MoE-50M (Shanghai AI Lab)	50	—	—	—	33.2%	40.3%	33.3%	38.6%
39	SVR (scikit-learn)	—	—	—	—	36.3%	35.6%	36.4%	36.6%
40	MOMENT-Small (CMU)	40	—	—	—	35.4%	35.4%	35.3%	38.2%
41	N-BEATS (Nixtla)	—	—	—	—	37.1%	35.2%	38.1%	33.2%
42	MOMENT-Base (CMU)	125	—	—	—	33.7%	37.1%	33.8%	37.8%
43	Seasonal Naive (Sktime)	—	—	—	—	33.3%	29.7%	31.9%	27.0%
44	PatchTST-Granite (IBM)	40	8.1%	8.1%	8.1%	27.3%	26.4%	27.2%	32.7%
45	N-HiTS (Nixtla)	—	—	—	—	7.5%	6.5%	7.1%	6.9%
46	Theta (Sktime)	—	—	—	—	3.6%	1.9%	3.6%	3.8%
47	Sundial (THUML)	128	0.0%	0.0%	0.0%	1.9%	1.9%	1.9%	1.9%

4.2 Skill Score Results

In Tables 6, 7, and 8 (Appendix B), we report the per-metric averaged skill scores, across univariate, multivariate, and covariate forecasting tasks, respectively. All skill scores compare model performance to a baseline model (Seasonal Naive). Positive skill scores indicate better performance than the baseline, while negative scores indicate worse performance.

Univariate (see Table 6).. The strongest models (TiRex, TiRex 1.1, TimesFM 2.5, Moirai 2.0, Chronos-2 and smaller Chronos variants) achieve roughly 0.4–0.66 skill on MAE, MAPE, and MASE—substantial geometric-mean improvement versus naive on those errors. TiRex leads MAE, MAPE, and MASE, while Chronos-2 leads RMSE, illustrating metric-dependent rankings. Many classical and lighter models remain above naive but with smaller skill (often about 0.1–0.4). Several foundation-model families (TTM, Time-MoE, MOMENT) and some baselines (e.g. XGBoost, Lag-Llama, PatchTST-Granite, Theta) fall below zero on most columns. Sundial exhibits extremely large negative skill, which is more suggestive of severe mismatch or unstable ratio behavior under clipping than of a mild underperformance relative to naive. TOTO illustrates heterogeneous metric behavior: decent skill on MAE, MAPE, and MASE but comparatively weak RMSE skill.

Multivariate (see Table 7).. TimesFM 2.5 attains the best skill in all four columns on this slice, with values around 0.64–0.68, slightly more consistent than in the univariate table. TiRex remains very competitive but does not win any column. Several models show mixed signs across metrics (e.g. Moirai 2.0 is positive on MAE, MASE, and RMSE but negative on MAPE; N-BEATS and TOTO show strongly negative MAPE while other metrics are mixed or positive), so MAPE-based conclusions should be drawn carefully. Moirai-Base, Moirai-Large, PatchTST-Granite, and Theta are deeply negative on multiple metrics; Sundial is again a large-magnitude outlier.

Covariate (see Table 8).. TabPFN-TS achieves the highest skill in every column (about 0.74–0.82), ahead of Chronos-2 (about 0.63–0.69), indicating particular strength when exogenous covariates are part of the task definition. TimesFM 200M also scores highly. Classical models such as ARIMA and Prophet occupy a strong mid-to-upper tier. As in the other slices, Time-MoE variants are clearly below naive on average for several metrics, and N-HiTS, Theta, and Sundial show extreme negative skill values that warrant separate discussion of failure modes rather than ranking alone.

5 Future Directions and Conclusion

In line with existing forecasting evaluation frameworks, in TempusBench we only consider benchmarks based on standard forecasting task features, such as target variate type, context length, and forecast length. Benchmarks could be defined more comprehensively in terms of hyperparameters that correspond to standard forecasting task features and others, such as task domains. Indeed, a more comprehensive set of benchmarks could test the per-domain (e.g., economics) performance of forecasting models across different target variate types, context lengths, and forecast lengths. In future work, we plan to develop a more flexible taxonomy for describing forecasting tasks.

Finally, we expect the datasets used to define our benchmarks will eventually be included in the pretaining corpora of TSFMs, as has been the case with many NLP benchmarks. To this end, in future work, we plan to develop dynamic benchmarks where test data is continuously refreshed. We have in mind two types of dynamic benchmarks: those based on synthetic data, where new datasets are periodically generated (e.g., seasonality tests), and those based on real-world data that update naturally over time (e.g., monthly inflation reports or daily weather observations).

References

[1] Gerald Woo, Chenghao Liu, Akshat Kumar, Caiming Xiong, Silvio Savarese, and Doyen Sahoo. Unified training of universal time series forecasting transformers. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 53140–53164, 2024. 1, 43

[2] Xu Liu, Juncheng Liu, Gerald Woo, Taha Aksu, Yuxuan Liang, Roger Zimmermann, Chenghao Liu, Silvio Savarese, Caiming Xiong, and Doyen Sahoo. Moirai-MoE: Empowering time series foundation models with sparse mixture of experts. *arXiv preprint arXiv:2410.10469*, 2024. 25

[3] Kashif Rasul, Arjun Ashok, Andrew Robert Williams, Hena Ghonia, Rishika Bhagwatkar, Arian Khorasani, Mohammad Javad Darvishi Bayazi, George Adamopoulos, Roland Riachi, Nadhir Hassen, et al. Lag-Llama: Towards foundation models for probabilistic time series forecasting. *arXiv preprint arXiv:2310.08278*, 2023. 27

[4] Abdul Fatir Ansari, Lorenzo Stella, Caner Turkmen, Xiyuan Zhang, Pedro Mercado, Huibin Shen, Oleksandr Shchur, Syama Sundar Rangapuram, Sebastian Pineda Arango, Shubham Kapoor, et al. Chronos: Learning the language of time series. *Transactions on Machine Learning Research*, 2024. 26, 43

[5] Abhimanyu Das, Weihao Kong, Rajat Sen, and Yichen Zhou. A decoder-only foundation model for time-series forecasting. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 10148–10167, 2024. 43

[6] Mononito Goswami, Konrad Szafer, Arjun Choudhry, Yifu Cai, Shuo Li, and Artur Dubrawski. Moment: A family of open time-series foundation models. *arXiv preprint arXiv:2402.03885*, 2024. 27, 43, 44, 46

[7] Andreas Auer, Patrick Podest, Daniel Klotz, Sebastian Böck, Günter Klambauer, and Sepp Hochreiter. TiRex: Zero-shot forecasting across long and short horizons with enhanced in-context learning. In *Advances in Neural Information Processing Systems*, pages 64179–64230, 2025. doi: 10.52202/085713-1929.

[8] Christoph Jürgen Hemmer and Daniel Durstewitz. True zero-shot inference of dynamical systems preserving long-term statistics. In *Advances in Neural Information Processing Systems*, pages 15139–15182, 2025. doi: 10.52202/085713-0455.

[9] Abdul Fatir Ansari, Oleksandr Shchur, Jaris Küken, Andreas Auer, Boran Han, Pedro Mercado, Syama Sundar Rangapuram, Huibin Shen, Lorenzo Stella, Xiyuan Zhang, et al. Chronos-2: From univariate to universal forecasting. *arXiv preprint arXiv:2510.15821*, 2025.

[10] Shi Bin Hoo, Samuel Müller, David Salinas, and Frank Hutter. From tables to time: Extending TabPFN-v2 to time series forecasting. *Transactions on Machine Learning Research*, 2026. 30, 31

[11] Kun Feng, Shaocheng Lan, Yuchen Fang, Wenchao He, Sihan Lu, Shuqi Gu, Xingyu Lu, Lintao Ma, and Kan Ren. Kairos: Toward adaptive and parameter-efficient time series foundation models. *arXiv preprint arXiv:2509.25826*, 2025.

[12] Vijay Ekambaram, Arindam Jati, Pankaj Dayama, Sumanta Mukherjee, Nam H Nguyen, Wesley M Gifford, Chandra Reddy, and Jayant Kalagnanam. Tiny time mixers (ttms): Fast pre-trained models for enhanced zero/few-shot forecasting of multivariate time series. *Advances in Neural Information Processing Systems*, 37:74147–74181, 2024. 31

[13] Ben Cohen, Emaad Khwaja, Youssef Doubli, Salahidine Lemaachi, Chris Lettieri, Charles Masson, Hugo Miccinilli, Elise Ramé, Qiqi Ren, Afshin Rostamizadeh, et al. This time is different: An observability perspective on time series foundation models. *arXiv preprint arXiv:2505.14766*, 2025. 27, 46, 47

[14] Yong Liu, Guo Qin, Zhiyuan Shi, Zhi Chen, Caiyin Yang, Xiangdong Huang, Jianmin Wang, and Mingsheng Long. Sundial: A family of highly capable time series foundation models. *arXiv preprint arXiv:2502.00816*, 2025. 1

[15] George E. P. Box and Gwilym M. Jenkins. *Time Series Analysis: Forecasting and Control*. Holden-Day, San Francisco, 1970. 1

[16] Vladimir N. Vapnik. *The Nature of Statistical Learning Theory*. Springer, New York, 1995. 1

[17] Harris Drucker, Christopher J. C. Burges, Linda Kaufman, Alexander Smola, and Vladimir Vapnik. Support vector regression machines. In *Advances in Neural Information Processing Systems 9*, pages 155–161. MIT Press, 1997. 1

[18] Rakshitha Godahewa, Christoph Bergmeir, Geoffrey I. Webb, et al. Monash time series forecasting archive. In *NeurIPS Datasets and Benchmarks Track*, 2021. 2, 3, 43

[19] Xiangfei Qiu, Jilin Hu, Lekui Zhou, Xingjian Wu, Junyang Du, Buang Zhang, Chenjuan Guo, Aoying Zhou, Christian S Jensen, Zhenli Sheng, and Bin Yang. TFB: Towards comprehensive and fair benchmarking of time series forecasting methods. *Proceedings of the VLDB Endowment*, 17:2363–2377, 2024. 3, 43

[20] Ailing Zeng, Muxi Chen, Lei Zhang, and Qiang Xu. Are transformers effective for time series forecasting? In *Proceedings of the AAAI conference on artificial intelligence*, volume 37, pages 11121–11128, 2023. 3, 43

[21] Zezhi Shao, Fei Wang, Yongjun Xu, Wei Wei, Chengqing Yu, Zhao Zhang, Di Yao, Tao Sun, Guangyin Jin, Xin Cao, et al. Exploring progress in multivariate time series forecasting: Comprehensive benchmarking and heterogeneity analysis. *IEEE Transactions on Knowledge and Data Engineering*, 37(1):291–305, 2025. doi: 10.1109/TKDE.2024.3484454. 3, 43

[22] Jiawen Zhang, Xumeng Wen, Zhenwei Zhang, Shun Zheng, Jia Li, and Jiang Bian. Probts: Benchmarking point and distributional forecasting across diverse prediction horizons. *Advances in Neural Information Processing Systems*, 37:48045–48082, 2024. 3

[23] Taha Aksu, Gerald Woo, Juncheng Liu, Xu Liu, Chenghao Liu, Silvio Savarese, Caiming Xiong, and Doyen Sahoo. Gift-eval: A benchmark for general time series forecasting model evaluation. *arXiv preprint arXiv:2410.10393*, 2024. 2, 3

[24] Yuxuan Liang, Haomin Wen, Yuqi Nie, Yushan Jiang, Ming Jin, Dongjin Song, Shirui Pan, and Qingsong Wen. Foundation models for time series analysis: A tutorial and survey. In *Proceedings of the 30th ACM SIGKDD conference on knowledge discovery and data mining*, pages 6555–6565, 2024. 2

[25] Prajakta S Kalekar et al. Time series forecasting using Holt-Winters exponential smoothing. *Kanwal Rekhi school of information Technology*, 4329008(13):1–13, 2004. 2, 33, 44, 45, 46

[26] Spyros Makridakis and Michele Hibon. The M3-competition: Results, conclusions and implications. *International Journal of Forecasting*, 16(4):451–476, 2000. 2, 43

[27] Spyros Makridakis, Evangelos Spiliotis, and Vassilis Assimakopoulos. The M4 competition: Results, findings, conclusion and way forward. *International Journal of Forecasting*, 34(4):802–808, 2018. 2, 43

[28] Pooja Anand, Mayank Sharma, and Anil Saroliya. A comparative analysis of artificial neural networks in time series forecasting using ARIMA vs Prophet. In *2024 International Conference on Communication, Computer Sciences and Engineering (IC3SE)*, pages 527–533. IEEE, 2024. 2, 33, 44, 45, 46, 47

[29] Salesforce. Gift-eval. Hugging Face Space, 2024. URL https://huggingface.co/spaces/Salesforce/GIFT-Eval. Accessed: 2025-08-29. 2

[30] u/nkafr. The rise of foundation time-series forecasting models. https://www.reddit.com/r/datascience/comments/1e865bt/the_rise_of_foundation_timeseries_forecasting/, 2024. URL https://www.reddit.com/r/datascience/comments/1e865bt/the_rise_of_foundation_timeseries_forecasting/. Reddit post on r/datascience. 2

[31] Rob J Hyndman and Anne B Koehler. Another look at measures of forecast accuracy. *International journal of forecasting*, 22(4):679–688, 2006. 2

[32] Rob J. Hyndman and George Athanasopoulos. *Forecasting: Principles and Practice*. OTexts, Melbourne, Australia, 3rd edition, 2021. URL https://otexts.com/fpp3/. 2

[33] Gerald Woo, Chenghao Liu, Akshat Kumar, Caiming Xiong, Silvio Savarese, and Doyen Sahoo. Unified training of universal time series forecasting transformers. *arXiv preprint arXiv:2402.02592*, 2024. 23

[34] Abhimanyu Das, Weihao Kong, Rajat Sen, and Yichen Zhou. A decoder-only foundation model for time-series forecasting. In *Forty-first International Conference on Machine Learning*, 2024. 25, 26

[35] Sima Siami-Namini and Akbar Siami Namin. Forecasting economics and financial time series: ARIMA vs. LSTM. *arXiv preprint arXiv:1803.06386*, 2018. 28, 32, 43, 44, 45, 46, 47

[36] Lin Lin, Fang Wang, Xiaolong Xie, and Shisheng Zhong. Random forests-based extreme learning machine ensemble for multi-regime time series prediction. *Expert Systems with Applications*, 83:164–176, 2017. 29

[37] Senyao Wang and Jin Ma. A novel ensemble model for load forecasting: Integrating random forest, XGBoost, and seasonal naive methods. In *2023 2nd Asian Conference on Frontiers of Power and Energy (ACFPE)*, pages 114–118. IEEE, 2023. 29, 35, 44, 45, 46

[38] Fan Zhang and Lauren J O'Donnell. Support vector regression. In *Machine learning*, pages 123–140. Elsevier, 2020. 30, 45

[39] Dimitrios D Thomakos and Konstantinos Nikolopoulos. Forecasting multivariate time series with the Theta method. *Journal of Forecasting*, 34(3):220–229, 2015. 34, 43, 45, 46

[40] Thomas R Willemain, Charles N Smart, Joseph H Shockor, and Philip A DeSautels. Forecasting intermittent demand in manufacturing: a comparative evaluation of Croston's method. *International Journal of forecasting*, 10(4):529–538, 1994. 35

[41] Indeed. Software Development Job Postings on Indeed in the United States [IHLIDXUSTPSOFTDEVE]. https://fred.stlouisfed.org/series/IHLIDXUSTPSOFTDEVE, 2025. Retrieved August 29, 2025. 36

[42] Coinbase. Coinbase Litecoin [CBLTCUSD]. https://fred.stlouisfed.org/series/CBLTCUSD, 2025. Retrieved August 29, 2025. 36

[43] U.S. Census Bureau. Manufacturers: Inventories to Sales Ratio [MNFCTRIRSA]. https://fred.stlouisfed.org/series/MNFCTRIRSA, 2025. Retrieved August 29, 2025. 36

[44] Bank for International Settlements. Real Residential Property Prices for Germany [QDER628BIS]. https://fred.stlouisfed.org/series/QDER628BIS, 2025. Retrieved August 29, 2025. 36

[45] Board of Governors of the Federal Reserve System (US). Federal Funds Effective Rate [FF]. https://fred.stlouisfed.org/series/FF, 2025. Retrieved August 29, 2025. 36

[46] U.S. Bureau of Economic Analysis. Personal Consumption Expenditures: Chain-type Price Index [DPCERG3A086NBEA]. https://fred.stlouisfed.org/series/DPCERG3A086NBEA, 2025. Retrieved August 29, 2025. 36

[47] SumanthVrao. Daily Climate Time Series Data. https://www.kaggle.com/datasets/sumanthvrao/daily-climate-time-series-data, 2019. Accessed on August 29, 2025. 36

[48] BITS Pilani - Goa. SplitSmart: An Open Dataset for Enabling Research in Energy-Efficient Ductless-Split Air Conditioner, 2024. Accessed on August 29, 2025. 36, 37

[49] U.S. Bureau of Labor Statistics. All Employees, Health Care [CES6562000101]. https://fred.stlouisfed.org/series/CES6562000101, 2025. Retrieved August 29, 2025. 36

[50] NoeyIsLearning. Soil and Environmental Monitoring. https://www.kaggle.com/datasets/noeyislearning/soil-and-environmental-monitoring, 2024. Accessed on August 29, 2025. 36, 37

[51] Jorge Bañuelos-Gimeno, Natalia Sobrino, and Rosa Arce-Ruiz. Initial Insights into Teleworking's Effect on Air Quality in Madrid City. *Environments*, 11(9):204, 2024. doi: 10.3390/environments11090204. URL https://www.mdpi.com/2076-3298/11/9/204. 36, 37

[52] RaminHuseyn. Web Traffic Dataset. https://www.kaggle.com/datasets/raminhuseyn/web-traffic-time-series-dataset, 2024. Accessed on August 29, 2025. 36

[53] UCI Machine Learning Repository. Hungarian Chickenpox Cases. https://doi.org/10.24432/C5103B, 2021. 36

[54] Alistair Johnson, Tom Pollard, and Roger Mark. MIMIC-III Clinical Database Demo (version 1.4). https://doi.org/10.13026/C2HM2Q, 2019. RRID:SCR_007345. 36

[55] Andrea Martiniano and Ricardo Ferreira. Absenteeism at work. https://doi.org/10.24432/C5X882, 2012. 36

[56] Daqing Chen. Online Retail. https://doi.org/10.24432/C5BW33, 2015. 36

[57] Adarsh Pal Singh and Sachin Chaudhari. Room Occupancy Estimation. https://doi.org/10.24432/C5P605, 2018. 36

[58] Nisarg Chodvadiya. Daily Gold Price (2015-2021) Time Series. https://www.kaggle.com/datasets/nisargchodavadiya/daily-gold-price-20152021-time-series, 2025. Accessed on August 29, 2025. 36, 37

[59] Ignacio Quintero González. 2001-2022 Hourly Dataset of Pollution in Madrid. https://www.kaggle.com/datasets/ignacioqg/20012022-hourly-dataset-of-pollution-in-madrid, 2022. Accessed on August 29, 2025. 36, 37

[60] DeltaTrup. LT 1-Minute Historical Stock Data (2003-2024). https://www.kaggle.com/datasets/deltatrup/lt-1-minute-historical-stock-data-2003-2024, May 2024. Accessed on August 29, 2025. 36

[61] Gabriel Santello. Airline Baggage Complaints – Time Series Dataset. https://www.kaggle.com/datasets/gabrielsantello/airline-baggage-complaints-time-series-dataset, 2023. Accessed on August 29, 2025. 37

[62] John McLennan, Greg Nash, Joseph Moore, Gosia Skowron, and Steven Woolsey. Utah FORGE: Well 16A(78)-32 Drilling Data. https://doi.org/10.15121/1776602, 2021. Accessed on August 29, 2025. 37

[63] New York City Department of Health and Mental Hygiene. COVID-19 daily counts of cases, hospitalizations, and deaths. https://catalog.data.gov/dataset/covid-19-daily-counts-of-cases-hospitalizations-and-deaths, 2025. NYC Open Data via Data.gov catalog. 37

[64] Riccardo Taormina et al. The Battle of the Attack Detection Algorithms: Disclosing Cyber Attacks on Water Distribution Networks. *Journal of Water Resources Planning and Management*, 144(8):04018048, aug 2018. doi: 10.1061/(ASCE)WR.1943-5452.0000969. URL https://www.batadal.net/data.html. 37

[65] Debashis74017. NIFTY-50 minute data. https://www.kaggle.com/datasets/debashis74017/nifty-50-minute-data, 2025. Kaggle dataset. 37

[66] Raj Kumar Pandey. Economy of the United States. https://www.kaggle.com/datasets/rajkumarpandey02/economy-of-the-united-states, 2023. Kaggle dataset. 37

[67] szrlee. DJIA 30 stock time series. https://www.kaggle.com/datasets/szrlee/stock-time-series-20050101-to-20171231, 2018. Kaggle dataset. 37, 38

[68] Dataset Engineer. Southern California energy consumption. https://www.kaggle.com/datasetengineer/southern-california-energy-consumption, 2024. Kaggle dataset. 37

[69] Data Science Lovers. Weather data. https://www.kaggle.com/datasets/rohitgrewal/weather-data, 2025. Kaggle dataset. 37

[70] Rohan Purohit. Monthly currency trends. https://www.kaggle.com/datasets/rohanpurohit0705/monthly-currency-trends, 2025. Kaggle dataset. 37, 38

[71] ziya07. Building performance dataset. https://www.kaggle.com/datasets/ziya07/construction-project-performance-dataset, 2024. Kaggle dataset. 38

[72] Zoya77. Solar irradiance and weather forecasting dataset. https://www.kaggle.com/datasets/zoya77/solar-irradiance-and-weather-forecasting-dataset, 2025. Kaggle dataset. 38

[73] Mikhail1681. Walmart sales. https://www.kaggle.com/datasets/mikhail1681/walmart-sales, 2024. Kaggle dataset. 38

[74] Dataset Engineer. CyberTec IIoT malware dataset (CIMD-2024). https://www.kaggle.com/datasets/datasetengineer/cybertec-iiot-malware-dataset-cimd-2024, 2025. Kaggle dataset. 38

[75] ziya07. Smart mobility traffic dataset. https://www.kaggle.com/datasets/ziya07/smart-mobility-traffic-dataset, 2025. Kaggle dataset. 38

[76] George E. P. Box, Gwilym M. Jenkins, Gregory C. Reinsel, and Greta M. Ljung. *Time Series Analysis: Forecasting and Control*. Wiley, 5 edition, 2015. 43

[77] Rob J. Hyndman, Anne B. Koehler, J. Keith Ord, and Ralph D. Snyder. *Forecasting with Exponential Smoothing: The State Space Approach*. Springer, 2008. 43

[78] Vassilis Assimakopoulos and Konstantinos Nikolopoulos. The Theta model: A decomposition approach to forecasting. *International Journal of Forecasting*, 16(4):521–530, 2000. 43

[79] Helmut Lütkepohl. *New Introduction to Multiple Time Series Analysis*. Springer, 2005. 43

[80] David Salinas, Valentin Flunkert, Jan Gasthaus, and Tim Januschowski. DeepAR: Probabilistic forecasting with autoregressive recurrent networks. *International Journal of Forecasting*, 36(3):1181–1191, 2020. doi: 10.1016/j.ijforecast.2019.07.001. 43

[81] Boris N. Oreshkin, Dmitri Carpov, Nicolas Chapados, and Yoshua Bengio. N-beats: Neural basis expansion analysis for interpretable time series forecasting. In *International Conference on Learning Representations (ICLR)*, 2020. 43

[82] Abhimanyu Das et al. Long-term forecasting with tide: Time-series dense encoder. In *Neural Information Processing Systems (NeurIPS)*, 2023. 43

[83] Bryan Lim, Sercan Ö. Arik, Nicolas Loeff, and Tomas Pfister. Temporal fusion transformers for interpretable multi-horizon time series forecasting, 2019. 43

[84] Yuqi Nie, Nam H. Nguyen, Phanwadee Sinthong, and Jayant Kalagnanam. A time series is worth 64 words: Long-term forecasting with transformers. In *International Conference on Learning Representations (ICLR)*, 2023. 43

[85] Han Liu et al. itransformer: Inverted transformers are effective for time series forecasting, 2023. 43

[86] Kashif Rasul, Calvin Seward, Ingmar Schuster, and Roland Vollgraf. Autoregressive denoising diffusion models for multivariate probabilistic time series forecasting. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 8857–8868, 2021. 43

[87] Yusuke Tashiro, Jiaming Song, Yang Song, and Stefano Ermon. Csdi: Conditional score-based diffusion models for probabilistic time series imputation. In *Neural Information Processing Systems (NeurIPS)*, 2021. 43

[88] Kashif Rasul, Abdul-Saboor Sheikh, Ingmar Schuster, Urs Bergmann, and Roland Vollgraf. Multivariate probabilistic time series forecasting via conditioned normalizing flows. In *International Conference on Learning Representations (ICLR)*, 2021. 43

[89] Kashif Rasul, Arjun Ashok, Andrew Robert Williams, Hena Ghonia, Rishika Bhagwatkar, Arian Khorasani, Mohammad Javad Darvishi Bayazi, George Adamopoulos, Roland Riachi, Nadhir Hassen, Marin Biloš, Sahil Garg, Anderson Schneider, Nicolas Chapados, Alexandre Drouin, Valentina Zantedeschi, Yuriy Nevmyvaka, and Irina Rish. Lag-llama: Towards foundation models for probabilistic time series forecasting, 2023. 43

[90] Yong Liu, Haoran Zhang, Chenyu Li, Xiangdong Huang, Jianmin Wang, and Mingsheng Long. Timer: Generative pre-trained transformers are large time series models. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 32369–32399, 2024. 43

[91] Shanghua Gao, Teddy Koker, Owen Queen, Thomas Hartvigsen, Theodoros Tsiligkaridis, and Marinka Zitnik. UniTS: A unified multi-task time series model. In *Advances in Neural Information Processing Systems*, volume 37, pages 140589–140631, 2024. doi: 10.52202/079017-4463. 43

[92] Vijay Ekambaram, Arindam Jati, Pankaj Dayama, Sumanta Mukherjee, Nam Nguyen, Wesley Gifford, Chandra Reddy, and Jayant Kalagnanam. Tiny time mixers (TTMs): Fast pre-trained models for enhanced zero/few-shot forecasting of multivariate time series. In *Advances in Neural Information Processing Systems*, volume 37, pages 74147–74181, 2024. doi: 10.52202/079017-2359. 43

[93] Mouxiang Chen, Lefei Shen, Zhuo Li, Xiaoyun Joy Wang, Jianling Sun, and Chenghao Liu. VisionTS: Visual masked autoencoders are free-lunch zero-shot time series forecasters. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 8979–9007, 2025. 43

[94] Spyros Makridakis, Evangelos Spiliotis, and Vassilios Assimakopoulos. M5 accuracy competition: Results, findings, and conclusions. *International Journal of Forecasting*, 38(4):1346–1364, 2022. doi: 10.1016/j.ijforecast.2021.11.013. 43

[95] Jiawen Zhang, Xumeng Wen, Zhenwei Zhang, Shun Zheng, Jia Li, and Jiang Bian. ProbTS: Benchmarking point and distributional forecasting across diverse prediction horizons. In *Advances in Neural Information Processing Systems*, volume 37, pages 48045–48082, 2024. doi: 10.52202/079017-1523. 43

[96] Sean J. Taylor and Benjamin Letham. Forecasting at scale. *The American Statistician*, 72(1):37–45, 2018. 43

[97] Markus Löning, Anthony Bagnall, Sajaysurya Ganesh, et al. sktime: A unified interface for machine learning with time series. In *Proceedings of the 2nd Workshop on Systems for ML (NeurIPS)*, 2019. 43

[98] Alexander Alexandrov et al. Gluonts: Probabilistic time series models in Python, 2020. 43

[99] Kashif Rasul. Pytorchts: A probabilistic deep learning library for time series, 2021. GitHub repository: https://github.com/zalandoresearch/pytorch-ts. 43

[100] Rob J. Hyndman and Anne B. Koehler. Another look at measures of forecast accuracy. *International Journal of Forecasting*, 22(4):679–688, 2006. 43

[101] Tilmann Gneiting and Adrian E. Raftery. Strictly proper scoring rules, prediction, and estimation. *Journal of the American Statistical Association*, 102(477):359–378, 2007. 43

[102] Raj Kumar Tamatta. *Time series forecasting of hospital Inpatients and Day case waiting list using ARIMA, TBATS and Neural Network Models*. PhD thesis, Dublin, National College of Ireland, 2018. 43

[103] Michael Boyles. 7 Financial Forecasting Methods to Predict Business Performance. https://online.hbs.edu/blog/post/financial-forecasting-methods, June 2022. Accessed on August 14, 2025. 44

[104] Leo Gao, Stella Biderman, Sid Black, Laurence Golding, Travis Hoppe, et al. The Pile: An 800GB dataset of diverse text for language modeling. *arXiv preprint arXiv:2101.00027*, 2020. 46

[105] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *2009 IEEE Conference on Computer Vision and Pattern Recognition*, pages 248–255. IEEE, 2009. 46

A Additional Mathematical Background

A.1 Mathematical notation

We adopt the following conventions to indicate the nature of the mathematical object at hand: We use calligraphic uppercase letters to denote sets (e.g., $\mathcal{X}$), bold uppercase letters to denote matrices (e.g., $\boldsymbol{X}$), bold lowercase letters to denote vectors (e.g., $\boldsymbol{p}$), lowercase letters to denote scalar quantities (e.g., x), and uppercase letters to denote random variables (e.g., X).

We denote the ith row vector of a matrix (e.g., $\boldsymbol{X}$) by the corresponding bold lowercase letter with subscript i (e.g., $\boldsymbol{x}_i$). Similarly, we denote the jth entry of a vector (e.g., $\boldsymbol{p}$ or $\boldsymbol{x}_i$) by the corresponding lowercase letter with subscript j (e.g., p_j or x_{ij}). We denote functions by a letter determined by the value of the function, e.g., f if the mapping is scalar valued, $\boldsymbol{f}$ if the mapping is vector valued, and $\mathcal{F}$ if the mapping is set valued.

We denote the set $\{1, \dots, n\}$ by $[n]$, the set $\{n, n+1, \dots, m\}$ by $[n : m]$, the set of natural numbers by $\mathbb{N}$, and the set of real numbers by $\mathbb{R}$. We denote the positive and strictly positive elements of a set using a $+$ or $++$ subscript, respectively, e.g., $\mathbb{R}_+$ and $\mathbb{R}_{++}$. For any $n \in \mathbb{N}$, we denote the n-dimensional vector of zeros and ones by $\mathbf{0}_n$ and $\mathbf{1}_n$, respectively.

A.2 Mathematical Definitions

We let $\Delta_n = \{\boldsymbol{x} \in \mathbb{R}_+^n \mid \sum_{i=1}^n x_i = 1\}$ denote the unit simplex in $\mathbb{R}^n$, and $\Delta(A)$ denote the set of all probability measures over a given set A. We also define the support of a probability density function $f \in \Delta(\mathcal{X})$ as $\operatorname{supp}(f) \doteq \{\boldsymbol{x} \in \mathcal{X} \mid f(\boldsymbol{x}) > 0\}$. Finally, we denote the orthogonal projection operator onto a set C by Π_C, i.e., $\Pi_C(\boldsymbol{x}) \doteq \arg\min_{\boldsymbol{y} \in C} \|\boldsymbol{x} - \boldsymbol{y}\|^2$.

A.3 Evaluation Metrics

An evaluation metric $\ell : \mathcal{Y}^h \times \mathcal{Y}^h \to \mathbb{R}_+$ is a positive-, scalar-valued function s.t. for any forecast $\hat{\boldsymbol{Y}} \in \mathcal{Y}^h$ and realized future target values $\boldsymbol{Y}^* \in \mathcal{Y}^h$, $\ell(\hat{\boldsymbol{Y}}, \boldsymbol{Y}^*) \geq 0$ denotes the distance between the forecast and the realized values. We consider the following evaluation metrics at present.

The *mean absolute error (MAE)* is defined as

$$\ell^{\text{MAE}}(\hat{\boldsymbol{Y}}, \boldsymbol{Y}^*) \doteq \frac{1}{mh} \sum_{i \in [m]} \sum_{t=1}^{h} |\hat{y}_{it} - y^*_{it}|$$

The *root mean squared error (RMSE)* is defined as

$$\ell^{\text{RMSE}}(\hat{\boldsymbol{Y}}, \boldsymbol{Y}^*) \doteq \sqrt{\frac{1}{mh} \sum_{i \in [m]} \sum_{t=1}^{h} (\hat{y}_{it} - y^*_{it})^2}$$

The *mean absolute scale error (MASE)* is defined as

$$\ell^{\text{MASE}}(\hat{\boldsymbol{Y}}, \boldsymbol{Y}^*) \doteq \frac{1}{mh} \sum_{i \in [m]} \sum_{t=1}^{h} \frac{|\hat{y}_{it} - y^*_{it}|}{\frac{1}{h-1} \sum_{t=1}^{l-1} |y_{it+1} - y_{it}|}$$

We note MAE is scale-dependent but less sensitive to outliers than MSE, which disproportionately penalizes large forecast errors. MASE normalizes errors w.r.t. the forecasts of a naive forecast method

(i.e., setting the next time step's forecast to be the current time step's realized value), making it scale-free and comparable across datasets and domains.

The *mean absolute percentage error (MAPE)* is defined as

$$\ell^{\mathrm{MAPE}}(\widehat{\boldsymbol{Y}}, \boldsymbol{Y}^*) \doteq \frac{100}{mh} \sum_{i \in [m]} \sum_{t=1}^{h} \frac{|\widehat{y}_{it} - y^*_{it}|}{|y^*_{it}|}$$

B Skill Score Results

Table 6. Per-metric averaged skill scores across univariate tasks

Rank	Model	MAE	MAPE	MASE	RMSE
1	TiRex (NX-AI)	**0.639**	**0.660**	**0.644**	0.596
2	TiRex 1.1 (NX-AI)	0.625	0.635	0.631	0.579
3	TimesFM 2.5 (Google)	0.622	0.640	0.628	0.584
4	Moirai 2.0 (Salesforce)	0.621	0.639	0.627	0.578
5	Chronos-2 (Amazon)	0.637	0.644	0.642	**0.608**
6	TimesFM 500M (Google)	0.525	0.563	0.532	0.482
7	PatchTST-FM (IBM)	0.506	0.518	0.513	0.475
8	Chronos-Bolt-Mini (Amazon)	0.551	0.581	0.558	0.517
9	TimesFM 200M (Google)	0.477	0.517	0.485	0.418
10	Chronos-Bolt-Tiny (Amazon)	0.544	0.570	0.551	0.510
11	Chronos-2-Small (Amazon)	0.623	0.631	0.629	0.592
12	Chronos-Bolt-Base (Amazon)	0.508	0.553	0.515	0.468
13	Chronos-Bolt-Small (Amazon)	0.526	0.557	0.534	0.492
14	Moirai-Base (Salesforce)	0.479	0.463	0.486	0.434
15	TabPFN-TS (Prior Labs)	0.516	0.536	0.523	0.463
16	Chronos-Large (Amazon)	0.437	0.510	0.445	0.389
17	Moirai-Large (Salesforce)	0.467	0.443	0.475	0.420
18	Chronos-Mini (Amazon)	0.438	0.517	0.446	0.391
19	Granite FlowState (IBM)	0.447	0.478	0.456	0.409
20	Moirai-Small (Salesforce)	0.430	0.408	0.438	0.399
21	Chronos-Tiny (Amazon)	0.417	0.500	0.426	0.371
22	N-HiTS (Nixtla)	0.432	0.433	0.415	0.354
23	Kairos-50M (Foundation Model Research)	0.459	0.498	0.468	0.422
24	Chronos-Small (Amazon)	0.378	0.423	0.387	0.341
25	N-BEATS (Nixtla)	0.378	0.421	0.387	0.332
26	TOTO (Datadog)	0.364	0.382	0.373	0.128
27	Kairos-10M (Foundation Model Research)	0.462	0.521	0.470	0.431
28	Chronos-Base (Amazon)	0.360	0.437	0.370	0.324
29	LSTM (Nixtla)	0.127	0.248	0.192	0.040
30	Moirai-MoE (Salesforce)	0.309	0.295	0.319	0.256
31	Kairos-23M (Foundation Model Research)	0.383	0.406	0.392	0.363
32	PatchTSMixer (IBM)	0.084	0.127	0.096	0.077
33	iTransformer (THUML)	0.226	0.215	0.237	0.171
34	SVR (scikit-learn)	0.172	0.217	0.185	0.146
35	Random Forest (scikit-learn)	0.124	0.172	0.137	0.123
36	ARIMA (Statsmodels)	0.172	0.217	0.155	0.138
37	Exponential Smoothing (Statsmodels)	0.093	0.083	0.099	0.049
38	Croston (Statsmodels)	0.049	0.106	0.039	0.015

Continued on next page

Table 6 — *continued from previous page*

Rank	Model	MAE	MAPE	MASE	RMSE
39	TTM R2 (IBM)	-0.114	-0.018	-0.097	-0.079
40	TTM R1 (IBM)	-0.117	-0.018	-0.100	-0.082
41	Time-MoE-200M (Shanghai AI Lab)	-0.145	-0.023	-0.127	-0.108
42	Time-MoE-50M (Shanghai AI Lab)	-0.145	-0.023	-0.127	-0.108
43	TTM R2.1 (IBM)	-0.119	-0.019	-0.102	-0.084
44	MOMENT-Large (CMU)	-0.118	-0.030	-0.102	-0.084
45	XGBoost (DMLC)	-0.106	-0.001	-0.056	-0.064
46	Prophet (Meta)	0.034	0.086	0.049	0.064
47	MOMENT-Base (CMU)	-0.127	-0.059	-0.110	-0.095
48	MOMENT-Small (CMU)	-0.138	-0.072	-0.121	-0.108
49	Seasonal Naive (Sktime)	0.000	0.000	0.000	0.000
50	Lag-Llama (Nixtla)	-0.650	-0.426	-0.626	-0.620
51	PatchTST-Granite (IBM)	-0.434	-0.457	-0.413	-0.371
52	Theta (Sktime)	-0.647	-0.632	-0.622	-0.570
53	Sundial (THUML)	-91.679	-83.721	-90.430	-96.323

Table 7. Per-metric averaged skill scores across multivariate tasks

Rank	Model	MAE	MAPE	MASE	RMSE
1	TiRex (NX-AI)	0.629	0.462	0.639	0.617
2	TiRex 1.1 (NX-AI)	0.624	0.494	0.635	0.605
3	TimesFM 2.5 (Google)	**0.653**	**0.681**	**0.663**	**0.644**
4	Chronos-2 (Amazon)	0.524	0.154	0.537	0.523
5	Chronos-2-Small (Amazon)	0.573	0.417	0.585	0.576
6	TimesFM 500M (Google)	0.577	0.264	0.589	0.563
7	PatchTST-FM (IBM)	0.565	0.169	0.577	0.572
8	TabPFN-TS (Prior Labs)	0.532	0.530	0.545	0.523
9	Chronos-Bolt-Mini (Amazon)	0.479	0.194	0.494	0.471
10	TimesFM 200M (Google)	0.602	0.250	0.613	0.606
11	Chronos-Bolt-Small (Amazon)	0.484	0.137	0.498	0.482
12	Chronos-Bolt-Base (Amazon)	0.468	0.202	0.483	0.466
13	Chronos-Bolt-Tiny (Amazon)	0.474	0.152	0.489	0.473
14	Moirai 2.0 (Salesforce)	0.368	-0.182	0.386	0.356
15	Kairos-50M (Foundation Model Research)	0.378	0.251	0.395	0.345
16	Kairos-23M (Foundation Model Research)	0.419	0.073	0.436	0.395
17	PatchTSMixer (IBM)	0.288	-0.255	0.283	0.299
18	Granite FlowState (IBM)	0.437	0.088	0.453	0.430
19	N-HiTS (Nixtla)	0.360	-0.344	0.379	0.385
20	Kairos-10M (Foundation Model Research)	0.357	0.091	0.375	0.350
21	Chronos-Mini (Amazon)	0.394	-0.003	0.411	0.347
22	Chronos-Base (Amazon)	0.434	-0.025	0.450	0.385

Continued on next page

Table 7 — *continued from previous page*

Rank	Model	MAE	MAPE	MASE	RMSE
23	N-BEATS (Nixtla)	0.296	-0.603	0.319	0.337
24	Chronos-Small (Amazon)	0.428	0.063	0.444	0.409
25	Chronos-Tiny (Amazon)	0.366	0.033	0.384	0.335
26	Chronos-Large (Amazon)	0.395	-0.007	0.412	0.355
27	TOTO (Datadog)	0.260	-0.876	0.281	-0.345
28	iTransformer (THUML)	0.297	-0.324	0.316	0.292
29	Moirai-Large (Salesforce)	-0.381	-2.120	-0.343	-1.358
30	SVR (scikit-learn)	-0.047	-0.143	-0.018	0.052
31	Prophet (Meta)	0.313	0.400	0.332	0.375
32	TTM R2 (IBM)	-0.140	0.203	-0.108	-0.096
33	TTM R2.1 (IBM)	-0.139	0.195	-0.107	-0.096
34	Croston (Statsmodels)	0.092	0.196	0.124	0.152
35	TTM R1 (IBM)	-0.140	0.201	-0.108	-0.096
36	Moirai-Base (Salesforce)	-0.905	-2.197	-0.852	-1.969
37	MOMENT-Large (CMU)	-0.142	-0.508	-0.110	-0.099
38	ARIMA (Statsmodels)	0.036	0.143	0.090	0.117
39	Random Forest (scikit-learn)	-0.088	-0.087	-0.057	-0.029
40	Time-MoE-200M (Shanghai AI Lab)	-0.383	0.063	-0.345	-0.279
41	Time-MoE-50M (Shanghai AI Lab)	-0.383	0.063	-0.345	-0.279
42	MOMENT-Base (CMU)	-0.156	-0.594	-0.123	-0.114
43	LSTM (Nixtla)	-0.157	-0.840	-0.065	-0.113
44	MOMENT-Small (CMU)	-0.169	-0.639	-0.136	-0.129
45	Moirai-Small (Salesforce)	0.091	-2.152	0.117	-0.156
46	Exponential Smoothing (Statsmodels)	-0.058	-0.099	-0.046	0.001
47	Seasonal Naive (Sktime)	0.000	0.000	0.000	0.000
48	PatchTST-Granite (IBM)	-0.483	-2.102	-0.442	-0.293
49	Theta (Sktime)	-1.640	-1.427	-1.566	-1.166
50	Sundial (THUML)	-93.691	-92.964	-93.798	-94.665

Table 8. Per-metric averaged skill scores across covariate tasks

Rank	Model	MAE	MAPE	MwASE	RMSE
1	Chronos-2 (Amazon)	0.630	0.688	0.638	0.628
2	TabPFN-TS (Prior Labs)	**0.757**	**0.824**	**0.762**	**0.742**
3	Chronos-2-Small (Amazon)	0.589	0.607	0.598	0.599
4	TimesFM 200M (Google)	0.716	0.775	0.722	0.704
5	Chronos-Bolt-Base (Amazon)	0.385	0.548	0.399	0.384
6	Chronos-Bolt-Tiny (Amazon)	0.417	0.575	0.430	0.411
7	Prophet (Meta)	0.657	0.690	0.665	0.670
8	Chronos-Bolt-Small (Amazon)	0.415	0.562	0.428	0.415
9	Chronos-Bolt-Mini (Amazon)	0.397	0.557	0.410	0.399

Continued on next page

Table 8 — *continued from previous page*

Rank	Model	MAE	MAPE	MASE	RMSE
10	TiRex (NX-AI)	0.412	0.556	0.425	0.391
11	TiRex 1.1 (NX-AI)	0.437	0.577	0.449	0.414
12	TimesFM 2.5 (Google)	0.381	0.529	0.395	0.371
13	PatchTST-FM (IBM)	0.338	0.491	0.352	0.369
14	Moirai 2.0 (Salesforce)	0.389	0.551	0.402	0.366
15	Kairos-50M (Foundation Model Research)	0.387	0.521	0.400	0.378
16	ARIMA (Statsmodels)	0.607	0.622	0.607	0.611
17	Kairos-10M (Foundation Model Research)	0.366	0.509	0.380	0.356
18	TimesFM 500M (Google)	0.341	0.491	0.356	0.333
19	Moirai-Small (Salesforce)	0.296	0.432	0.311	0.318
20	Kairos-23M (Foundation Model Research)	0.338	0.478	0.353	0.328
21	Chronos-Mini (Amazon)	0.360	0.538	0.374	0.346
22	PatchTSMixer (IBM)	0.167	0.371	0.183	0.226
23	Granite FlowState (IBM)	0.345	0.478	0.359	0.335
24	TOTO (Datadog)	0.106	0.255	0.125	-0.099
25	Chronos-Tiny (Amazon)	0.329	0.465	0.344	0.309
26	Chronos-Base (Amazon)	0.240	0.405	0.256	0.269
27	Exponential Smoothing (Statsmodels)	0.023	0.202	0.047	0.084
28	Chronos-Small (Amazon)	0.275	0.447	0.291	0.275
29	Moirai-Base (Salesforce)	0.187	0.301	0.205	0.220
30	LSTM (Nixtla)	0.043	0.319	0.068	-0.015
31	Croston (Statsmodels)	0.125	0.238	0.153	0.184
32	TTM R2 (IBM)	-0.285	0.066	-0.257	-0.151
33	TTM R1 (IBM)	-0.285	0.068	-0.257	-0.151
34	MOMENT-Large (CMU)	-0.286	0.056	-0.258	-0.153
35	iTransformer (THUML)	0.162	0.186	0.181	0.216
36	Random Forest (scikit-learn)	-0.252	-0.018	-0.225	-0.139
37	Time-MoE-200M (Shanghai AI Lab)	-0.998	-0.499	-0.955	-0.775
38	Time-MoE-50M (Shanghai AI Lab)	-0.998	-0.499	-0.955	-0.775
39	SVR (scikit-learn)	-0.207	-0.020	-0.180	-0.155
40	MOMENT-Small (CMU)	-0.302	-0.047	-0.274	-0.177
41	N-BEATS (Nixtla)	-0.080	-0.081	-0.041	-0.091
42	MOMENT-Base (CMU)	-0.299	-0.022	-0.270	-0.168
43	Seasonal Naive (Sktime)	0.000	0.000	0.000	0.000
44	PatchTST-Granite (IBM)	-0.219	-0.112	-0.193	-0.063
45	N-HiTS (Nixtla)	-12.563	-14.671	-12.588	-12.257
46	Theta (Sktime)	-36.257	-39.329	-35.930	-32.343
47	Sundial (THUML)	-94.735	-94.791	-94.822	-98.339

C Forecasters

Table 9. Summary of forecasters included in TempusBench.

Category	Included Models	Core Characteristics
Foundation Models	*Moirai, Moirai-MoE, TimesFM, TimesFM-2.0, Chronos, Lag-Llama, Toto, MOMENT*	**Paradigm:** Universal, zero-shot/few-shot forecasting. A single large model is pre-trained on massive, diverse datasets and generalizes to new tasks without redataset-specific training. **Architecture:** Primarily based on Transformers or other deep learning structures like MLP-Mixers. They process raw time series data via patching or novel tokenization schemes. **I/O:** Natively handle univariate, multivariate, and covariate data. Often produce probabilistic forecasts.
Classic Machine Learning	*LSTM, Random Forest (RF), XGBoost, SVR, TabPFN-TS, Tiny Time Mixers (TTM)*	**Paradigm:** Supervised learning models trained per-dataset. They excel at capturing complex, non-linear relationships but require specific training for each task. **Architecture:** Diverse. E.g., RNNs (for sequence memory), Tree Ensembles (for interaction effects), and Kernel Methods. **I/O:** Typically require explicit feature (e.g., lags, calendar variables) engineering to create a tabular format. Most often produce point forecasts.
Statistical & Decomposable	*ARIMA, Holt-Winters, Prophet, Theta Method, Croston's Method, Seasonal Naive*	**Paradigm:** Assume the time series is generated by an underlying statistical process or can be decomposed into simpler, interpretable components like trend and seasonality. **Architecture:** An explicit mathematical formula is fit directly to an individual time series. **I/O:** Highly interpretable point forecasts. Often specialized for particular data patterns (e.g., intermittency with *Croston's*).

In this section, we summarize the forecasting models which have been included in TempusBench. We summarize all models in Table 9, and provide and comparison of TSFMs, machine learning forecasting models, and statistical forecasting models in Table 10.

Moirai. Moirai [33] is a universal time-series forcasting model developed by Salesforce AI Research, built upon a masked encoder-only Transformer architecture. It is designed as a single, large pre-trained

Table 10. Comparative Overview of Forecasting Models

Feature	Group 1: Foundation Models	Group 2: Machine Learning Models	Group 3: Classical & Statistical Models
Models Included	■ *Moirai / Moirai-MoE* ■ *TimesFM / TimesFM-2.0* ■ *Chronos* ■ *Lag-Llama* ■ *Toto* ■ *MOMENT*	■ *LSTM* ■ *Random Forest (RF)* ■ *XGBoost* ■ *SVR* ■ *TabPFN-TS* ■ *Tiny Time Mixers (TTM)*	■ *ARIMA* ■ *Holt-Winters* ■ *Prophet* ■ *Theta Method* ■ *Croston's Method* ■ *Seasonal Naive*
Core Paradigm	Large, pre-trained models designed for universal, zero-shot forecasting. They learn general time-series patterns from massive, diverse datasets.	Models trained for a specific forecasting task, often relying on feature engineering. They leverage distinct architectures (e.g., recurrence, ensembles) rather than massive pre-training.	Model-based approaches assuming an underlying stochastic process or decomposable structure. Parameters are estimated directly from the target time series.
Architecture	Primarily Transformer-based (Encoder, Decoder, or both). Innovations include MoE layers, residual forecasting, and specialized attention mechanisms.	Diverse architectures: MLP-Mixer (*TTM*), RNN (*LSTM*), Tabular-Transformer (*TabPFN-TS*), Tree Ensembles (*RF, XGBoost*), and Kernel-based (*SVR*).	Mathematical formulations: State-space models (*ARIMA, Holt-Winters*), decomposable additive models (*Prophet, Theta*), and simple heuristics (*Croston's, Seasonal Naive*).
Input Handling	Process raw time series, typically via patching (*Moirai, TimesFM*), lag-based tokenization (*Lag-Llama*), or value quantization (*Chronos*). Can natively handle uni/multivariate series.	Generally require explicit feature (e.g., lags, calendar variables) engineering to create a tabular dataset (*RF, XGBoost, SVR*). *LSTM* and *TTM* process raw sequences.	Operate directly on the univariate time series. May require stationarity (*ARIMA*) or be specialized for patterns like seasonality (*Holt-Winters*) or intermittency (*Croston's*).
Output Type	Mostly probabilistic, predicting the parameters of a flexible distribution. Point forecasts are derived from the distribution (e.g., median).	Primarily point forecasts. *TabPFN-TS* is a notable exception, providing a probabilistic output by approximating the posterior.	Primarily point forecasts. *Prophet* is an exception, generating uncertainty intervals. Probabilistic versions exist but are not standard.
Key Trait	Powerful zero-shot/few-shot performance. High model capacity and ability to generalize across domains without dataset-specific training.	Model-specific strengths: computational efficiency (*TTM*), modeling long-term dependencies (*LSTM*), and capturing complex non-linear interactions (*RF, XGBoost*).	High interpretability and strong statistical foundations. Often specialized and highly efficient for specific data patterns (e.g., trend, seasonality, intermittency).

model capable of handling diverse forecasting tasks without dataset-specific retraining. The model is pre-trained on LOTSA, a large-scale archive of over 27 billion observations, enabling it to perform powerful zero-shot forecasting.

- **Input:** Accepts univariate or multivariate time series with an arbitrary number of variates and covariates.
- **Output:** Produces a probabilistic forecast by predicting the parameters of a flexible mixture distribution (composed of Student's t, Negative Binomial, Log-Normal, and low-variance Normal distributions) for the next token in an autoregressive manner.
- **Architecture:** Employs a masked encoder-only Transformer. Its key innovations include:
  - **Multi Patch Size Projection:** Uses different patch sizes to effectively process time series of varying frequencies.
  - **Any-variate Attention:** Flattens multivariate series into a single sequence and uses binary attention biases to manage an arbitrary number of variates while maintaining permutation equivariance.
- **Forecasting Type:** A universal, zero-shot, probabilistic forecaster. It can generate point forecasts by taking the median of the predicted distribution.

Moirai-MoE. Moirai-MoE [2] is an advanced version of the Moirai foundation model that integrates a Sparse Mixture of Experts (MoE) architecture. Instead of relying on heuristic-based, frequency-specific projection layers, Moirai-MoE delegates the task of modeling diverse time series patterns to specialized "expert" networks within its Transformer layers. This design allows for automatic, token-level specialization in a data-driven manner, leading to improved accuracy and fewer activated parameters, resulting in faster inference.

- **Input:** Accepts univariate or multivariate time series with an arbitrary number of variates and covariates.
- **Output:** Produces a probabilistic forecast by predicting the parameters of a flexible mixture distribution (composed of Student's t, Negative Binomial, Log-Normal, and low-variance Normal distributions) for the next token in an autoregressive manner.
- **Architecture:** Employs a decoder-only Transformer that replaces the standard Feed-Forward Network (FFN) layers with MoE layers. Key architectural changes from the original Moirai include:
  - **Mixture of Experts (MoE):** A gating function routes each time series token to a small subset of specialized expert networks, allowing the model to handle diverse patterns at a granular level.
  - **Single Projection Layer:** It uses a single input/output projection layer for all time series, removing the dependency on frequency-based heuristics.
- **Forecasting Type:** A universal, zero-shot, probabilistic forecaster that is more accurate and efficient (in terms of activated parameters) than the original Moirai model. It can generate point forecasts by taking the median of the predicted distribution.

TimesFM. TimesFM [34] is a time-series foundation model developed by Google Research, designed for zero-shot forecasting. It is based on a decoder-only Transformer architecture and is pretrained on a very large corpus of time-series data, combining both real-world and synthetic sources. The model's key objective is to provide accurate out-of-the-box point forecasts on unseen datasets without requiring any dataset-specific training.

- **Input:** Accepts a univariate time series context window.

- **Output:** Produces a point forecast for a given prediction horizon.
- **Architecture:** Employs a decoder-only Transformer architecture that processes the time series in patches. Key architectural features include:
  - **Decoder-Only Transformer:** Utilizes a standard decoder-style attention mechanism to autoregressively predict future values patch by patch.
  - **Input Patching:** The input time series is segmented into non-overlapping patches, which are then embedded using a residual block of MLPs before being fed to the Transformer.
- **Forecasting Type:** A universal, zero-shot, point forecaster designed primarily for long-horizon forecasting tasks.

TimesFM-2.0. TimesFM-2.0 [34] is an improved version of the original foundation model from Google Research. While retaining the same decoder-only Transformer architecture, its key innovation lies in forecasting the residual component of a time series after performing a seasonal-trend decomposition. This approach makes the model significantly more accurate, particularly for time series that exhibit clear trends.

- **Input:** Accepts a univariate time series context window.
- **Output:** Produces a point forecast for a given prediction horizon.
- **Architecture:** Based on the original decoder-only Transformer with input patching. The primary architectural update is its residual forecasting methodology:
  - **Seasonal-Trend Decomposition:** The model first decomposes the input series to separate its trend and seasonal components.
  - **Residual Forecasting:** The core Transformer then forecasts the residual (the signal remaining after decomposition). This forecast is added back to the projected trend to produce the final prediction.
- **Forecasting Type:** A universal, zero-shot, point forecaster with enhanced performance on non-stationary time series compared to its predecessor.

Chronos. Chronos [4] is a family of pretrained time series models developed by Amazon Science that frames forecasting as a language modeling task. The core idea is to "tokenize" time series values by scaling and quantizing them into a fixed vocabulary. By doing so, standard Transformer-based language model architectures can be trained on sequences of these tokens using a cross-entropy loss, effectively learning the "language" of time series.

- **Input:** Accepts a univariate time series context window.
- **Output:** Produces a probabilistic forecast by generating multiple sample future trajectories. A point forecast can be derived from the median of these samples.
- **Architecture:** Based on standard language model architectures (specifically the T5 encoder-decoder family). Its defining characteristic is its unique data preprocessing pipeline:
  - **Tokenization via Quantization:** The model first applies mean scaling to the input time series. It then quantizes these scaled values into a finite set of discrete tokens, converting the continuous series into a sequence of categorical variables.
  - **Language Model Training:** The model is trained to predict the next token in a sequence using a standard cross-entropy loss, analogous to how a language model predicts the next word.
- **Forecasting Type:** A universal, zero-shot, probabilistic forecaster.

Lag-Llama. [3] Lag-Llama is a foundation model for univariate probabilistic time-series forecasting. It is built upon a decoder-only Transformer architecture, similar to LLaMA, and is pretrained on a large, diverse corpus of open-source time-series data. The model's key innovation is its tokenization strategy, which uses lagged values of the time series as input features, allowing it to generalize across different frequencies and domains.

- **Input:** Accepts a univariate time series context window.
- **Output:** Produces a probabilistic forecast by outputting the parameters of a Student's t-distribution for the next time step. Future trajectories are generated autoregressively.
- **Architecture:** Based on a decoder-only Transformer (LLaMA). Its defining characteristic is its input representation:
  - **Tokenization via Lag Features:** Instead of patching, the input token for each time step is a vector composed of lagged values from the time series history (e.g., values from 1, 7, and 14 days prior). These values are augmented with standard date-time features.
  - **Value Scaling:** Applies robust scaling (using median and IQR) to normalize the input values and includes the scaling parameters as additional features.
- **Forecasting Type:** A universal, zero-shot/few-shot, probabilistic forecaster.

Toto. Toto (Time Series Optimized Transformer for Observability) [13] is a foundation model from Datadog, specifically designed for multivariate time-series forcasting with a focus on observability data (i.e., latency, throughput, etc). It is built on a decoder-only Transformer architecture and incorporates several novel components to handle the unique challenges of observability data, such as high non-stationarity and heavy-tailed distributions. The model is pretrained on a large and diverse corpus that includes real-world observability data, public datasets, and synthetic data.

- **Input:** Accepts multivariate time series.
- **Output:** Produces a probabilistic forecast by predicting the parameters of a Student's T mixture model.
- **Architecture:** A decoder-only Transformer with several key innovations tailored for observability data:
  - **Patch-based Causal Normalization:** A novel per-patch scaling method that computes normalization statistics from current and past data to handle highly nonstationary series.
  - **Proportional Factorized Attention:** An efficient attention mechanism that uses a mix of time-wise and variate-wise attention blocks to judiciously model interactions in high-dimensional multivariate data.
  - **Student's T Mixture Model Head:** An output layer that models the predictive distribution as a mixture of Student's T distributions to better capture the complex, heavy-tailed nature of observability metrics.
  - **Composite Robust Loss:** A hybrid loss function combining negative log-likelihood with a robust point-wise loss to stabilize training in the presence of outliers.
- **Forecasting Type:** A universal, zero-shot, probabilistic forecaster for multivariate time series.

MOMENT. MOMENT (Multi-task, Open-source, Foundation Model for Time-series) [6] is a family of open-source foundation models from Carnegie Mellon University designed for general-purpose time series analysis. The models are built on a Transformer encoder architecture and are pretrained on a large, diverse collection of public time series called the "Time Series Pile." A key characteristic of

MOMENT is its versatility; it is designed to serve as a building block for a wide range of downstream tasks, including forecasting, classification, anomaly detection, and imputation, often with minimal task-specific fine-tuning.

- **Input:** Accepts a univariate time series of a fixed length. Multivariate time series are handled by treating each channel independently.
- **Output:** Produces a reconstructed version of the input time series. This output can be adapted for various downstream tasks, such as generating forecasts by masking future values or extracting embeddings for classification.
- **Architecture:** A standard Transformer encoder that processes time-series data in patches.
  - **Masked Pre-training:** The model is pretrained using a masked time series prediction task. It learns to reconstruct randomly masked patches of the input time series, enabling it to learn robust representations.
  - **Patching:** The input time series is segmented into non-overlapping patches, which are then linearly projected into embeddings for the Transformer.
  - **Lightweight Prediction Head:** A simple linear layer is used to reconstruct the time series from the Transformer's output embeddings. This head can be easily replaced or adapted for different downstream tasks.
- **Forecasting Type:** A universal foundation model for general time series analysis. It can be used for zero-shot or few-shot forecasting (point-based), classification, anomaly detection, and imputation.

Long Short-Term Memory (LSTM). Long Short-Term Memory (LSTM) is a type of Recurrent Neural Network (RNN) architecture specifically designed to learn long-term dependencies in sequential data. Unlike traditional neural networks, LSTMs have internal mechanisms called "gates" that regulate the flow of information. These gates enable the network to selectively remember or forget information over long periods, making it particularly well-suited for time-series forecasting. [35]

- **Input:** A time-series dataset.
- **Output:** A point forecast for one or more future time steps.
- **Mathematical Formulation:** The core of an LSTM unit is its cell state, $\widehat{\boldsymbol{c}}_t$, which acts as a memory. The flow of information into and out of the cell is controlled by three gates: the forget gate ($\boldsymbol{f}_t$), the input gate ($\boldsymbol{i}_t$), and the output gate ($\boldsymbol{o}_t$). At each time step t, these gates update the cell state and produce a hidden state, $\widehat{\boldsymbol{h}}_t$.

$$\text{Forget Gate:} \quad \boldsymbol{f}_t = \sigma(\boldsymbol{w}_f \cdot [\widehat{\boldsymbol{h}}_{t-1}, \boldsymbol{y}_t] + b_f) \tag{6}$$

$$\text{Input Gate:} \quad \boldsymbol{i}_t = \sigma(\boldsymbol{w}_i \cdot [\widehat{\boldsymbol{h}}_{t-1}, \boldsymbol{y}_t] + b_i) \tag{7}$$

$$\text{Candidate State:} \quad \widetilde{\boldsymbol{c}}_t = \tanh(\boldsymbol{w}_c \cdot [\widehat{\boldsymbol{h}}_{t-1}, \boldsymbol{y}_t] + b_c) \tag{8}$$

$$\text{Cell State Update:} \quad \widehat{\boldsymbol{c}}_t = \boldsymbol{f}_t \odot \widehat{\boldsymbol{c}}_{t-1} + \boldsymbol{i}_t \odot \widetilde{\boldsymbol{c}}_t \tag{9}$$

$$\text{Output Gate:} \quad \boldsymbol{o}_t = \sigma(\boldsymbol{w}_o \cdot [\widehat{\boldsymbol{h}}_{t-1}, \boldsymbol{y}_t] + b_o) \tag{10}$$

$$\text{Hidden State Update:} \quad \widehat{\boldsymbol{h}}_t = \boldsymbol{o}_t \odot \tanh(\widehat{\boldsymbol{c}}_t) \tag{11}$$

where $\boldsymbol{W}$ and $\boldsymbol{b}$ are the weight matrices and bias vectors for each gate, σ is the sigmoid function, and $\odot$ denotes element-wise multiplication. The final prediction is typically generated by passing the hidden state $\widehat{\boldsymbol{h}}_t$ through a dense output layer.

- **Forecasting Type:** A neural network model for point forecasting that can capture complex non-linear patterns in time-series data.

Random Forest. Random Forest is an ensemble machine learning model that operates by constructing a multitude of decision trees at training time. In time-series forcasting, it is applied as a regression model to a featurized dataset. By fitting numerous trees on various sub-samples of the data and employing randomness in feature selection, it improves predictive accuracy and controls over-fitting. The final prediction is an average of the outputs from all individual trees, making the model robust and capable of capturing complex, non-linear relationships. [36]

- **Input:** A feature matrix $\boldsymbol{X}$ where rows are observations and columns are engineered features (e.g., lags, calendar variables), and a corresponding target vector $\boldsymbol{y}$.
- **Output:** A point forecast for each input feature vector.
- **Architecture and Formulation:** A Random Forest is an ensemble of B decision trees. Its predictive power comes from two sources of randomness introduced during training:
  - **Bagging (Bootstrap Aggregating):** Each individual tree, f_b, is trained on a bootstrap sample (a random sample drawn with replacement) from the original training dataset.
  - **Feature Randomness:** When splitting a node in a tree, the algorithm considers only a random subset of the total features, which decorrelates the trees in the forest.

  For a new input feature vector $\boldsymbol{x}$, the forecast is the average of the predictions from all B trees in the ensemble:

$$\hat{\boldsymbol{y}}(\boldsymbol{x}) = \frac{1}{B}\sum_{b=1}^{B} f_b(\boldsymbol{x}) \tag{12}$$

- **Forecasting Type:** An ensemble machine learning model for point forecasting. It is non-parametric and highly effective at modeling non-linear relationships between features and the target variable.

XGBoost. XGBoost (Extreme Gradient Boosting) is a powerful and efficient implementation of the gradient boosting framework. It is an ensemble model that builds decision trees sequentially, where each new tree is trained to correct the errors made by the previous ones. For time-series forcasting, XGBoost is used as a regression model on a featurized dataset, making it highly effective at capturing complex, non-linear relationships between the engineered features (e.g., lags, calendar variables) and the target. [37]

- **Input:** A feature matrix $\boldsymbol{X}$ and a corresponding target vector $\boldsymbol{y}$.
- **Output:** A point forecast for each input feature vector.
- **Architecture and Formulation:** XGBoost builds an additive model where the final prediction is the sum of the predictions from K decision trees:

$$\hat{\boldsymbol{y}}_i = \sum_{k=1}^{K} f_k(\boldsymbol{x}_i) \tag{13}$$

  The trees are added one at a time in a greedy fashion. The k-th tree, f_k, is chosen to minimize a regularized objective function:

$$\mathcal{L}^{(k)} = \sum_{i=1}^{n} l(\boldsymbol{y}_i, \hat{\boldsymbol{y}}_i^{(k-1)} + f_k(\boldsymbol{x}_i)) + \Omega(f_k) \tag{14}$$

where l is a differentiable loss function, $\hat{\boldsymbol{y}}_i^{(k-1)}$ is the prediction from the first $k-1$ trees, and Ω is a regularization term that penalizes the complexity of the tree:

$$\Omega(f) = \gamma T + \frac{1}{2}\lambda \sum_{j=1}^{T} w_j^2 \tag{15}$$

Here, T is the number of leaves in the tree, $\boldsymbol{w}$ is the vector of scores on the leaves, and γ and λ are regularization parameters.

- **Forecasting Type:** An ensemble machine learning model for point forecasting, known for its high performance, speed, and regularization capabilities.

Support Vector Regression (SVR). Support Vector Regression (SVR) is a supervised learning algorithm that extends the principles of Support Vector Machines (SVMs) to regression problems. Instead of finding a hyperplane that separates classes, SVR aims to find a function that deviates from the target values by a value no greater than a specified margin, ϵ, for as many of the training points as possible. It is particularly effective in high-dimensional spaces and is robust to some outliers due to its use of an ϵ-insensitive loss function, which ignores errors within this margin. [38]

- **Input:** A feature matrix $\boldsymbol{X}$ and a corresponding target vector $\boldsymbol{y}$.
- **Output:** A point forecast for each input feature vector.
- **Mathematical Formulation:** The goal of SVR is to find a function $f(\boldsymbol{x}) = \boldsymbol{w}^T\boldsymbol{x} + b$ that is as "flat" as possible. This is achieved by minimizing the norm of the weight vector, $||\boldsymbol{w}||^2$. The optimization problem is formulated to tolerate errors up to a margin ϵ while penalizing points that fall outside this margin using slack variables ξ_i and ξ_i^*. The primal optimization problem is:

$$\min_{\boldsymbol{w},b,\boldsymbol{\xi}} \frac{1}{2}||\boldsymbol{w}||^2 + C\sum_{i=1}^{n}(\xi_i + \xi_i^*) \tag{16}$$

subject to the constraints:

$$\boldsymbol{y}_i - (\boldsymbol{w}^T\boldsymbol{x}_i + b) \leq \epsilon + \xi_i \tag{17}$$

$$(\boldsymbol{w}^T\boldsymbol{x}_i + b) - \boldsymbol{y}_i \leq \epsilon + \xi_i^* \tag{18}$$

$$\xi_i, \xi_i^* \geq 0 \tag{19}$$

Here, C is a regularization parameter that controls the trade-off between the flatness of the model and the amount up to which deviations larger than ϵ are tolerated. Non-linear relationships are handled by mapping the data to a higher-dimensional space using a kernel function.

- **Forecasting Type:** A machine learning model for point forecasting that is robust to some outliers and effective in high-dimensional feature spaces.

TabPFN. TabPFN [10] is a prior-fitted network (PFN) for tabular classification and regression. It treats supervised learning on tabular data as in-context learning: rather than fitting parameters to a new dataset at test time, TabPFN amortizes training by meta-learning a posterior predictive distribution over datasets sampled from a structured prior (a causal graphical model over synthetic tabular data). At inference, the entire training set is passed as context via a transformer, and predictions for test points are produced in a single forward pass.

- **Input:** A tabular dataset (training split) provided as in-context examples, alongside test rows to be predicted. Each row consists of numerical or categorical features with no assumed temporal structure.
- **Output:** A predictive distribution over class labels (classification) or real-valued targets (regression); point predictions are obtained from the distribution mode or mean.
- **Architecture:**
  - *Prior:* Training data is synthesized from a Bayesian network prior (structural causal models based on graph and noise distributions, and their transformations), enabling meta-training without training on real-world datasets.
  - *Transformer Backbone:* A standard transformer encodes the concatenated context (training rows with labels) and query (test rows), attending jointly over examples and features to approximate the posterior predictive.
  - *Prediction Head:* A lightweight output head maps transformer representations to a continuous distribution, which are post-hoc discretized if the predictand is discrete.
- **Forecasting Type:** A zero-shot, amortized tabular predictor; not natively a time-series model, though downstream extensions (e.g., TabPFN-TS) adapt TABPFN to forecasting by featurizing time-series windows into tabular form.

TabPFN-TS. TabPFN-TS [10] is a novel approach that adapts TabPFN-v2, a general-purpose tabular foundation model, for time-series forecasting. The core methodology involves recasting the forecasting problem as a tabular regression task. This objective is achieved through lightweight feature engineering on the time index, without relying on lagged values. Notably, the underlying TabPFN-v2 model was pretrained exclusively on synthetic tabular data and has not seen any time-series data.

- **Input:** A univariate time series, which is converted into a feature matrix based on timestamps.
- **Output:** Produces a probabilistic forecast by approximating the posterior predictive distribution for each future time step. Point forecasts can be derived from the mean or median of this distribution.
- **Architecture:** It does not use a time-series-specific architecture. Instead, it relies on:
  - **Feature Engineering:** The time series is transformed into a tabular dataset by creating features from timestamps: e.g., standard calendar features (hour of day, day of week, etc.), automatically detected seasonal features via a Fourier transform, and a simple running index.
  - **TabPFN-v2 Model:** The generated tabular data is fed into the pretrained TabPFN-v2 model, which performs the regression task to predict future values.
- **Forecasting Type:** A universal, zero-shot, probabilistic forecaster.

Tiny Time Mixers (TTM). Tiny Time Mixers (TTM) [12] is a family of lightweight pre-trained models from IBM Research, based on the efficient TSMixer architecture. In contrast to large, LLM-based approaches, TTMs are designed to be extremely small (<1M parameters) and fast, while still providing strong zero-shot and few-shot forecasting performance. The models are pre-trained exclusively on a large corpus of public time-series datasets, making them a highly efficient alternative for universal forecasting.

- **Input:** Accepts univariate or multivariate time series, with optional support for exogenous variables during the fine-tuning stage.
- **Output:** Produces a point forecast for a given prediction horizon.

- **Architecture:** Based on the MLP-Mixer architecture. The model is pre-trained in a channel-independent manner and uses a multi-level structure to handle diverse data and tasks.
  - **TSMixer Backbone:** The core of the model uses simple MLP blocks for temporal and feature mixing, avoiding the computational overhead of Transformer-based attention.
  - **Multi-Resolution Pre-training:** Employs several novel techniques to handle heterogeneous datasets, including adaptive patching (using different patch configurations at different layers) and data augmentation via downsampling (creating additional training examples by resampling at coarser time resolutions.).
  - **Multi-level Modeling:** Uses a frozen pre-trained backbone and a smaller, fine-tunable decoder, which can incorporate channel-mixing and an **exogenous mixer** to fuse external signals for target-specific tasks.
- **Forecasting Type:** A universal, zero-shot/few-shot, point forecaster, notable for its small size and computational efficiency.

ARIMA. The Autoregressive Integrated Moving Average (ARIMA) [35] model is a class of statistical models for analyzing and forecasting time-series data. It is a generalization of the simpler Autoregressive Moving Average (ARMA) model that can be applied to non-stationary time series. The model's name reflects its three core components: Autoregression (AR), Integrated (I), and Moving Average (MA). These components capture the key temporal structures within the data, such as dependencies on past observations and past forecast errors.

- **Input:** A univariate time series.
- **Output:** A point forecast for future time steps. While classical ARIMA produces point forecasts, probabilistic forecasts can be generated by assuming a distribution for the error term.
- **Mathematical Formulation:** An ARIMA(p, d, q) model is defined by three parameters: the order of the autoregressive component (p), the degree of differencing (d), and the order of the moving average component (q). The model assumes that the differenced time series, $\widetilde{\boldsymbol{y}}_t = (1 - B)^d \boldsymbol{y}_t$, is stationary, where B is the backshift operator. The formulation for the stationary series $\widetilde{\boldsymbol{y}}_t$ is:

$$\widetilde{\boldsymbol{y}}_t = c + \sum_{i=1}^{p} \phi_i \widetilde{\boldsymbol{y}}_{t-i} + \sum_{j=1}^{q} \theta_j \epsilon_{t-j} + \epsilon_t \tag{20}$$

  where:
  - p is the autoregressive order, representing the number of lagged observations included in the model.
  - d is the degree of differencing, representing the number of times the raw observations are differenced to achieve stationarity.
  - q is the moving average order, representing the size of the moving average window applied to past forecast errors.
  - $\boldsymbol{\phi}$ is the vector of autoregressive coefficients.
  - $\boldsymbol{\theta}$ is the vector of moving average coefficients.
  - c is a constant term.
  - ϵ_t is the white noise error term at time t, typically assumed to be drawn from a Gaussian distribution with zero mean.

- **Forecasting Type:** A statistical model that provides point forecasts. It is often used as a baseline in forecasting tasks. Seasonal variations can be included by using a Seasonal ARIMA (SARIMA) model.

Holt-Winters Exponential Smoothing. Holt-Winters is an extension of exponential smoothing that explicitly models trend and seasonality. It is a widely used statistical method for forecasting time-series data that exhibit these components. The method operates by applying exponential smoothing to three components: the level, the trend, and the seasonality. There are two primary variations of the model, additive and multiplicative, which differ in how they incorporate the seasonal component. [25]

- **Input:** A univariate time series with trend and seasonality.
- **Output:** A point forecast for future time steps.
- **Mathematical Formulation:** The model provides separate updating equations for the level ($\widehat{l}_t$), trend ($\widehat{b}_t$), and seasonal ($\widehat{s}_t$) components, using smoothing parameters α, β, and γ, respectively. Let L be the length of the seasonal period.
  Additive Method: Used when the seasonal variation is roughly constant throughout the series.

$$\text{Level:} \quad \widehat{l}_t = \alpha(\boldsymbol{y}_t - \widehat{s}_{t-L}) + (1-\alpha)(\widehat{l}_{t-1} + \widehat{b}_{t-1}) \tag{21}$$

$$\text{Trend:} \quad \widehat{b}_t = \beta(\widehat{l}_t - \widehat{l}_{t-1}) + (1-\beta)\widehat{b}_{t-1} \tag{22}$$

$$\text{Seasonality:} \quad \widehat{s}_t = \gamma(\boldsymbol{y}_t - \widehat{l}_t) + (1-\gamma)\widehat{s}_{t-L} \tag{23}$$

The forecast for h steps ahead from time t, denoted $\widehat{\boldsymbol{y}}_{t+h|t}$, is:

$$\widehat{\boldsymbol{y}}_{t+h|t} = \widehat{l}_t + h\widehat{b}_t + \widehat{s}_{t-L+h_L^+} \quad \text{where } h_L^+ = \lfloor (h-1) \bmod L \rfloor + 1 \tag{24}$$

Multiplicative Method: Used when the seasonal variation changes in proportion to the level of the series.

$$\text{Level:} \quad \widehat{l}_t = \alpha\left(\frac{\boldsymbol{y}_t}{\widehat{s}_{t-L}}\right) + (1-\alpha)(\widehat{l}_{t-1} + \widehat{b}_{t-1}) \tag{25}$$

$$\text{Trend:} \quad \widehat{b}_t = \beta(\widehat{l}_t - \widehat{l}_{t-1}) + (1-\beta)\widehat{b}_{t-1} \tag{26}$$

$$\text{Seasonality:} \quad \widehat{s}_t = \gamma\left(\frac{\boldsymbol{y}_t}{\widehat{l}_t}\right) + (1-\gamma)\widehat{s}_{t-L} \tag{27}$$

The forecast for h steps ahead is given by:

$$\widehat{\boldsymbol{y}}_{t+h|t} = (\widehat{l}_t + h\widehat{b}_t)\widehat{s}_{t-L+h_L^+} \quad \text{where } h_L^+ = \lfloor (h-1) \bmod L \rfloor + 1 \tag{28}$$

- **Forecasting Type:** A statistical model for point forecasting that can handle various combinations of trend and seasonality.

Prophet. Prophet is a forecasting procedure developed by Meta, based on a decomposable time series model. It is designed to be robust to missing data and shifts in the trend, and it typically handles holidays and seasonal effects well. The model fits an additive model with components for trend, seasonality, and holidays. [28]

- **Input:** A univariate time series with timestamps.
- **Output:** A point forecast, along with uncertainty intervals.

- **Mathematical Formulation:** The Prophet model is specified as a sum of three components:

$$\boldsymbol{y}_t = g(t) + s(t) + h(t) + \epsilon_t \tag{29}$$

where:

- $g(t)$ is the trend component, which is modeled as either a piecewise linear or logistic growth function. This allows the model to capture non-periodic changes in the time series.
- $s(t)$ is the seasonality component, which models periodic changes (e.g., yearly, weekly, daily). It is approximated by a Fourier series:

$$s(t) = \sum_{n=1}^{N} \left(a_n \cos\left(\frac{2\pi nt}{P}\right) + b_n \sin\left(\frac{2\pi nt}{P}\right) \right) \tag{30}$$

where P is the period of the seasonality (e.g., 365.25 for yearly).

- $h(t)$ is the holiday component, which represents the effects of holidays and special events. It is modeled as a sum of indicator functions for each holiday.
- ϵ_t is the error term, assumed to be normally distributed white noise.

- **Forecasting Type:** A decomposable statistical model for point and probabilistic forecasting, particularly effective for business time series with strong seasonal patterns and holiday effects.

Theta Method. The Theta method is a statistical forecasting technique based on the concept of decomposition. It models a time series by breaking it down into two components, or "theta lines." The first line represents the long-term trend of the data, while the second line is constructed to capture the short-term dynamics by modifying the curvature of the original series (i.e., by computing a linear function of the second difference of the time-series)). These two lines are forecasted independently and then combined to produce the final forecast. The standard Theta model has been shown to be equivalent to Simple Exponential Smoothing with a drift term. [39]

- **Input:** A univariate time series.
- **Output:** A point forecast for future time steps.
- **Mathematical Formulation:** The method decomposes the original time series, $\boldsymbol{y}_t$, into two theta lines.
  - **Line 1 (Trend Component):** This line is the simple linear trend fit to the data, which is found by ordinary least squares regression:

$$\widetilde{\boldsymbol{y}}_t^{(1)} = \widehat{a} + \widehat{b}t \tag{31}$$

  This line is extrapolated linearly to produce a forecast.
  - **Line 2 (Short-term Component):** This line is constructed by modifying the original series with a coefficient θ. A common and effective choice is $\theta = 2$, which doubles the local curvatures of the series. This modified series, $\widetilde{\boldsymbol{y}}_t^{(2)}$, is then forecast using Simple Exponential Smoothing (SES).

  The final forecast, $\widehat{\boldsymbol{y}}_{t+h}$, is a simple average of the forecasts from the two lines:

$$\widehat{\boldsymbol{y}}_{t+h} = \frac{1}{2}\left(\widehat{\boldsymbol{y}}_{t+h}^{(1)} + \widehat{\boldsymbol{y}}_{t+h}^{(2)}\right) \tag{32}$$

- **Forecasting Type:** A statistical decomposition model for point forecasting, often used as a strong baseline for its simplicity and performance.

Croston's Method. Croston's method [40] is a forecasting technique specifically designed for intermittent demand time series, which are characterized by sporadic, non-zero values interspersed with periods of zero demand. The method decomposes the original time series into two separate components: the magnitude of the non-zero demand and the time interval between consecutive demands. By forecasting these two components separately using Simple Exponential Smoothing and then combining them, the model provides a more accurate estimate of the mean demand per period compared to standard smoothing methods, which can be biased when applied to intermittent data.

- **Input:** A univariate time series with intermittent demand.
- **Output:** A point forecast for the average demand per period.
- **Mathematical Formulation:** The method maintains and updates two estimates: one for the non-zero demand size ($\hat{\boldsymbol{z}}$) and one for the interval between demands ($\hat{\boldsymbol{p}}$). Let α be a smoothing parameter, let $\boldsymbol{y}_t$ be the demand at time t, and let q be the time elapsed since the last demand. An update occurs only when a non-zero demand is observed ($\boldsymbol{y}_t > 0$):

$$\hat{\boldsymbol{z}}_t = \hat{\boldsymbol{z}}_{t-1} + \alpha(\boldsymbol{y}_t - \hat{\boldsymbol{z}}_{t-1}) \tag{33}$$

$$\hat{\boldsymbol{p}}_t = \hat{\boldsymbol{p}}_{t-1} + \alpha(q - \hat{\boldsymbol{p}}_{t-1}) \tag{34}$$

  If demand at time t is zero, the estimates are not updated ($\hat{\boldsymbol{z}}_t = \hat{\boldsymbol{z}}_{t-1}$, $\hat{\boldsymbol{p}}_t = \hat{\boldsymbol{p}}_{t-1}$) and the interval counter q is incremented. After a demand occurs, q is reset to 1. The final forecast for the mean demand per period, $\hat{\boldsymbol{y}}_t$, is the ratio of the two smoothed components:

$$\hat{\boldsymbol{y}}_t = \frac{\hat{\boldsymbol{z}}_t}{\hat{\boldsymbol{p}}_t} \tag{35}$$

- **Forecasting Type:** A statistical model for point forecasting, specialized for intermittent or "lumpy" demand patterns.

Seasonal Naive. The Seasonal Naive model is a simple yet effective baseline method for forecasting time series with a strong seasonal component. Its core principle is that the forecast for a future period is equal to the last observed value from the same season. For example, as long as the season has not changed, the forecast for this Monday would be the value from last Monday. Despite its simplicity, it serves as a crucial benchmark for more complex models. [37]

- **Input:** A univariate time series with a known seasonal period.
- **Output:** A point forecast for future time steps.
- **Mathematical Formulation:** The forecast for h steps ahead from time t, denoted $\hat{\boldsymbol{y}}_{t+h|t}$, is the last observed value from the corresponding season. Let L be the seasonal period (e.g., $L = 7$ for daily data with weekly seasonality). The forecast is given by:

$$\hat{\boldsymbol{y}}_{t+h|t} = \boldsymbol{y}_{t+h-L\cdot k} \tag{36}$$

  where $k = \lceil h/L \rceil$ is an integer that ensures the lagged time index refers to the most recent observation from the target season. For a one-season-ahead forecast ($h = L$), this simplifies to $\hat{\boldsymbol{y}}_{t+L|t} = \boldsymbol{y}_t$.
- **Forecasting Type:** A simple statistical baseline for seasonal point forecasting.

D Benchmark Tasks included in TempusBench

This section describes the datasets for each benchmark task, as summarized in Table 2.

Notation. l: context window; h: forecast horizon (from each task's `task.yaml`). n: number of target variables; m: number of covariate variables (CSV rows with `variable_type` `target` vs. `covariate`). **Tot. pts.**: total number of scalar observations $T(n+m)$ for the full stored series, with T the common time length.

Table 11. Summary of **univariate** benchmark tasks

Category	Task	l	h	n	m	Tot. pts.
Trend						
Non-stationary	Software job postings [41]	512	64	1	0	1,827
Decomposition						
Additive	Synthetic additive (Section D.1)	1024	64	1	0	3,000
Multiplicative	Synthetic multiplicative (Section D.2)	1024	64	1	0	3,000
Frequency						
Days	Coinbase Litecoin [42]	512	64	1	0	1,827
Months	Inventories/sales [43]	64	30	1	0	402
Quarters	German house prices [44]	32	17	1	0	221
Weeks	Federal funds rate [45]	1024	64	1	0	3,713
Years	Personal consumption [46]	32	5	1	0	96
Seasonality						
Periodic	Synthetic cyclic	1024	64	1	0	3,000
Quasiperiodic	Synthetic non-stationary	1024	64	1	0	3,000
Domain						
Climate	Delhi climate [47]	512	64	1	0	1,462
Economics/Finance	Coinbase Litecoin [42]	512	64	1	0	1,827
Energy	Room SplitSmart [48]	128	39	1	0	561
Healthcare	Employees health care [49]	64	33	1	0	427
Manufacturing	Inventories/sales [43]	64	30	1	0	402
Nature	Soil monitoring [50]	32	8	1	0	679
Sales	German house prices [44]	32	17	1	0	221
Software	Software job postings [41]	512	64	1	0	1,827
Transport	Madrid BEN pollution [51]	2048	64	1	0	172,622
Web	Web traffic [52]	1024	64	1	0	2,793
Data sparsity						
Dense	Chickenpox [53]	128	35	1	0	522
Sparse	Patient chart [54]	12	4	1	0	8,093
Value type						
Binary	Absenteeism [55]	128	55	1	0	740
Categorical	Online retail [56]	2048	64	1	0	541,909
Count	Occupancy [57]	450	64	1	0	10,129

Table 12. Summary of **multivariate** benchmark tasks

Category	Task	l	h	n	m	Tot. pts.
Frequency						
Days	Gold India [58]	1024	64	5	0	20,120
Hours	Madrid transport pollution [59]	2048	64	14	0	2,544,542
Minutes	U.S. stocks2003–24 [60]	2048	64	6	0	732,660
Minutes	U.S. stocks 2003–24 (longest) [60]	2048	64	6	0	732,660

Continued on next page

Table 12 — *continued from previous page*

Category	Task	l	h	n	m	Tot. pts.
Months	Airlines baggage [61]	32	8	4	0	336
Manufacturing	Utah drilling [62]	2048	64	35	0	338,135
Seasonality						
Periodic	Madrid transport (cyclical) [59]	2048	64	14	0	2,544,542
Domain						
Economics/Finance	Gold India [58]	1024	64	5	0	20,120
Economics/Finance	Gold India (real) [58]	1024	64	5	0	20,120
Energy	Room SplitSmart [48]	2048	64	2	0	21,206
Healthcare	NYC Covid [63]	512	64	54	0	108,270
Nature	Soil monitoring [50]	1024	64	127	0	549,021
Nature	Soil monitoring (500) [50]	1024	64	127	0	549,021
Sales	Airlines baggage [61]	32	8	4	0	336
Software	Water-network attacks [64]	512	64	44	0	76,604
Transport	Airlines baggage (100) [61]	32	8	4	0	336
Transport	Madrid BEN pollution [51]	2048	64	14	0	2,544,542
Transport	Madrid BEN pollution (noisy) [59]	2048	64	14	0	2,544,542
Data sparsity						
Dense	Gold India [58]	1024	64	5	0	20,120
Value type						
Continuous	Gold India [58]	1024	64	5	0	20,120
Count	Madrid BEN pollution [51]	2048	64	14	0	2,544,542

Table 13. Summary of **covariate** benchmark tasks

Category	Task	l	h	n	m	Tot. pts.
Trend						
Non-stationary	NIFTY-50 minute covariates (non-stationary) [65]	2048	64	1	4	2,001,355
Noisy data	U.S. macro covariates (high-variation panel) [66]	18	5	1	6	336
Frequency						
Days	U.S. equities (daily covariates) [67]	2048	64	1	4	15,095
Hours	SoCal energy (hourly covariates) [68]	2048	64	1	34	1,840,475
Minutes	NIFTY-50 minute sector covariates [65]	2048	64	1	4	2,001,355
longest	Long weather covariate streams [69]	2048	64	1	4	2,001,355
Months	Monthly FX covariates [70]	200	16	1	3	1,248
Years	U.S. macro (annual covariates) [66]	18	5	1	6	336
Seasonality						
Cyclical	Weather covariates (diurnal/seasonal) [69]	2048	64	1	5	52,704
Domain						
Climate	Multi-year weather/climate covariates [69]	2048	64	1	5	52,704
Economics/Finance	U.S. equities (econ./finance covs.) [67]	2048	64	1	4	15,095
real	U.S. equities (real-return covs.) [67]	2048	64	1	4	15,095
Energy	SoCal energy consumption covariates [68]	2048	64	1	34	1,840,475
Healthcare	NYC Covid (covariate design) [63]	1536	42	1	53	108,270

Continued on next page

Table 13 — *continued from previous page*

Category	Task	l	h	n	m	Tot. pts.
Manufacturing	Construction performance covariates [71]	2048	64	1	15	800,000
∼500	Solar/weather (wide cov. field) [72]	256	22	3	9	6,000
∼100	Solar/weather (moderate field) [72]	32	6	3	9	1,200
Nature	Solar irradiance + weather covariates [72]	256	64	3	9	12,000
Sales	Walmart retail covariates [73]	45	16	45	225	38,610
Software	Cybertec IIoT malware (software telemetry) [74]	2048	64	1	25	1,177,514
Transport	Smart mobility covariates [75]	2048	64	1	10	55,000
Data sparsity						
Dense	Monthly FX (dense cov. panel) [70]	200	16	1	3	1,248
Value type						
Continuous	U.S. equities (continuous covs.) [67]	2048	64	1	4	15,095
Count	Cybertec IIoT (count covariates) [74]	2048	64	1	25	1,177,514

D.1 Synthetic Data: Cyclic Seasonality with Additive Trends

In this section, we describe how we construct synthetic data with cyclic seasonality using additive trends only.

D.1.1 Description

This category of synthetic data models a time series that exhibits both a complex seasonal pattern and a persistent, long-term trend. These data are generated using two related methods. Both start with a foundational signal that combines multi-frequency sinusoids with a linear trend. The second, more complex method builds upon this foundation by introducing an additional, randomized sinusoidal component to the signal.

In both cases, non-negative noise from an exponential distribution is added to the deterministic signal. These datasets are ideal for testing a model's ability to identify and separate periodicities from an underlying linear trend, with the second method providing a more complex seasonal structure.

D.1.2 Mathematical Formulation

The generation process for both methods is based on a primary signal, $y_{\text{base}}(t)$, which includes seasonal, trend, and offset components:

$$y_{\text{base}}(t) = \underbrace{2\sin(t) + 2\cos\left(\frac{t}{2}\right)}_{\text{Seasonality}} + \underbrace{\frac{1}{4}t}_{\text{Trend}} + \underbrace{4}_{\text{Offset}} \tag{37}$$

D.1.3 Method 1: Fixed Additive Trend

In the first method, the true signal, $y_1(t)$, is simply the base function. The final observed value, Y_t, is this signal plus an additive noise term, ϵ_t.

$$Y_t = y_1(t) + \epsilon_t = y_{\text{base}}(t) + \epsilon_t \tag{38}$$

D.1.4 Method 2: Randomized Additive Trend

The second method introduces additional complexity. For each generated time series, a random frequency parameter, α, is sampled once from a continuous uniform distribution:

$$\alpha \sim U(a, b) \tag{39}$$

In the provided code, this range is fixed from $a = 0$ to $b = 5$. This parameter is used to create an additional sinusoidal component that is added to the base signal. The true signal, $y_2(t)$, is therefore:

$$y_2(t) = y_{\text{base}}(t) + \sin(\alpha t) \tag{40}$$

The final observed value, Y_t, is this enhanced signal plus the noise term:

$$Y_t = y_2(t) + \epsilon_t \tag{41}$$

D.1.5 Noise Model

In both of the aforementioned methods, the noise term, ϵ_t, is drawn from an exponential distribution with a scale parameter β:

$$\epsilon_t \sim \text{Exponential}(\beta) \tag{42}$$

D.1.6 Adjustable Parameters

The data generation process is controlled by the following parameters.

- **Number of Points (`num_points`, N):** This integer parameter sets the total number of data points, defining the length of the time series.
- **Start Time (`start_time`, t_0):** This parameter defines the initial time value for the series.
- **Noise Scale (`noise_std`, β):** This parameter represents the scale (and mean) of the exponential noise distribution. A larger value of β increases the average magnitude of the positive noise added to the base signal.
- **Random Frequency (`alpha`, α):** (Method 2 only) This parameter is not set by the user, but is sampled internally from a uniform distribution $U(0, 5)$ for each generated series. It introduces variability in the seasonal component across different datasets created using the second method.

D.2 Synthetic Data: Cyclic Seasonality with Multiplicative and Additive Trends

In this section, we describe how we construct synthetic data with cyclic seasonality based on an amalgamation of additive and multiplicative trends.

D.2.1 Description

This category of synthetic data models a time series characterized by a complex interaction of seasonal components and trends. A key feature is a multiplicative trend, where the amplitude of one of the seasonal components grows exponentially over time. This is combined with another stable seasonal component and a linear additive trend.

These data are generated using two related methods. The first method uses a fixed, deterministic signal. The second method introduces additional complexity by adding another sinusoidal component with a randomized frequency to the base signal. In both cases, non-negative noise from an exponential distribution is added. These datasets are particularly useful for testing a model's ability to handle het-

eroskedasticity, where the variance of the series changes over time, in the presence of other seasonalities and trends.

D.2.2 Mathematical Formulation

Both methods are built upon a primary signal, $y_{\text{base}}(t)$, which is a composite of several functions:

$$y_{\text{base}}(t) = \underbrace{e^{t/100}\sin(t)}_{\text{Multiplicative Seasonality}} + \underbrace{3\cos\left(\frac{t}{2}\right)}_{\text{Additive Seasonality}} + \underbrace{\frac{1}{2}t}_{\text{Linear Trend}} \tag{43}$$

D.2.3 Method 1: Fixed Multiplicative Trend

In the first method, the true signal, $y_1(t)$, is simply the base function. The final observed value, Y_t, is this signal plus an additive noise term, ϵ_t.

$$Y_t = y_1(t) + \epsilon_t = y_{\text{base}}(t) + \epsilon_t \tag{44}$$

D.2.4 Method 2: Randomized Additive Component

This method is idential to the Randomized Additive Component in Section D.1.4.

D.2.5 Noise Model

The noise model is identical to the noise model in Section D.1.5.

D.2.6 Adjustable Parameters

The adjustable parameters are identical to those of Section D.1.6.

E Benchmark Evaluations

All results for all metrics, tasks, and models are available on the TempusBench webpage. You can inspect the interactive leaderboard and download a ZIP archive. That bundle includes, for each task type (univariate, multivariate, covariate), leaderboard, per-metric win-rate, and per-metric skill-score tables in ready-made LaTeX and CSV files under `latex/` and `csv/`, which can be used as needed. In addition, per-task tables are displayed in the Task Explorer.

Analysis of Univariate Task Performance

The strongest overall win rates in that snapshot are led by **TiRex** (NX-AI, ∼300M parameters) at **72.6%**, followed by **TiRex 1.1** at **71.3%**. **TimesFM 2.5** (Google, ∼200M) is third at **70.3%**, then **Moirai 2.0** (Salesforce, ∼12M) at **67.4%**, and **Chronos-2** (Amazon, ∼120M) at **65.8%**. Additional foundation checkpoints in the same top tier include **TimesFM 500M** (64.8%), **PatchTST-FM** (IBM, 63.4%), and compact **Chronos-Bolt** variants (e.g., Mini63.2%, Tiny 63.0%, Base 61.8%).

On **pooled averages** of the scalar columns (mean over all task×window rows feeding that model), **TiRex 1.1** attains the best displayed MAE (0.418), CRPS (0.252), quantile score (0.126), and weighted interval score (0.252). **TimesFM 2.5** achieves the best pooled RMSE (0.556). **Chronos-Tiny** obtains the lowest pooled MAPE among models in the table for this slice; MAPE is not comparable across tasks without normalization, so task-level exports should be read before drawing fine-grained conclusions. Pooled MASE can be dominated by a few heavy-tailed tasks; use per-task MASE from the downloadable CSV rather than the column average alone.

Analysis of Multivariate Task Performance

TiRex leads win rate at **82.8%**, narrowly followed by **TiRex 1.1** at **82.0%**. **TimesFM 2.5** is third (**79.2%**), **Chronos-2** fourth (**77.7%**), and **Chronos-2-Small** (40M) fifth (**74.8%**). **TimesFM 500M**, **PatchTST-FM**, and **TabPFN-TS** (Prior Labs, ∼1M parameters) occupy the next ranks at **73.1%**, **72.5%**, and **66.6%** respectively—highlighting that a billion-scale parameter gap is not required to stay in the upper win-rate band on this slice.

For pooled CRPS, quantile score, and WIS, **TiRex 1.1** is best on average; it also leads pooled MAE (0.336) and RMSE (0.548). **TimesFM 2.5** achieves the best pooled MAPE and MASE among models in the table for this snapshot.

Comparison of univariate and multivariate results

As **TiRex** and **TiRex 1.1** achieve the top-two win rates on both univariate and multivariate data, with **TimesFM 2.5** and **Chronos-2** in the top five for both, suggesting that the best univariate checkpointsmodels also scale to jointmultivariate modeling. Covariate tasks reshuffle leaders, however; Chronos-family models move up.

Analysis of Covariate Task Performance

Win-rate order differs on this task than univariate and multivariate: **Chronos-2** places first at **79.3%**, with **TabPFN-TS** second at **78.6%** despite its small footprint. **Chronos-2-Small** is third (**74.8%**), then

TimesFM 200M (**70.2%**) and **Chronos-Bolt-Base** (**66.0%**). **Prophet** (Meta) achieves a **65.3%** win rate, illustrating that classical baselines stay relevant when exogenous regressors are explicit.

On pooled averages, **Chronos-2** leads CRPS, quantile score, WIS, and RMSE; **TabPFN-TS** leads MAE and MASE. **Chronos-Bolt-Tiny** shows the lowest pooled MAPE in this slice.

F Additional Related Work

Classical time-series forecasting began with statistical models that exploit stochastic structure and domain priors, including ARIMA and its Box–Jenkins methodology [76], exponential-smoothing state-space ETS [77], the Theta method [78], and multivariate VAR models [79]. Deep learning methods later advanced accuracy and scale by learning nonlinear temporal dependencies from large corpora: DeepAR [80], N-BEATS [81], DLinear [20], TiDE [82], TFT [83], PatchTST [84], and iTransformer [85]. Probabilistic forecasters further model predictive distributions, e.g., diffusion-based TimeGrad [86], score-based CSDI for imputation and forecasting [87], and conditional-flow GRU-NVP [88].

TSFMs.. Inspired by NLP/vision pretraining, TSFMs train on heterogeneous corpora and evaluate in zero/few-shot settings across domains and horizons. Representative models include Moirai [1], Chronos [4], TimesFM [5], Lag-Llama [89], Timer [90], UniTS [91], TTM (Tiny Time Mixers) [92], Moment [6], and multimodal VisionTS [93]. Collectively, they demonstrate strong zero-shot point and probabilistic accuracy on diverse benchmarks while revealing open challenges at long horizons (error accumulation) and at very high frequencies.

Public datasets and repositories.. Public corpora have underpinned progress from statistical to foundation-model eras. The M-competitions (M3 and M4) provided broad univariate benchmarks across domains and frequencies [26, 27], followed by the retail-demand M5 competition [94]. The Monash Time-Series Forecasting Archive curates a large, standardized repository spanning many domains and sampling granularities [18]. Large-scale pretraining/evaluation collections include LOTSA (released with Moirai) [1], the Chronos corpus with in-domain/zero-shot splits [4], and the diverse univariate corpus aggregated in Lag-Llama [89]. Task-focused collections such as the LTSF suite [20] (e.g., ETT datasets) and broader benchmarks like TFB [19] and ProbTS [95] assemble datasets emphasizing horizon length, covariates, and probabilistic outputs.

Evaluation frameworks and benchmarks.. Tooling and standardized evaluation have evolved in parallel. Practitioner libraries such as Prophet [96] and sktime [97] offer classical and ML baselines with unified interfaces, while GluonTS [98] and PyTorchTS [99] provide probabilistic deep-learning pipelines. Benchmarking efforts including LTSF [20], BasicTS+ [21] , TFB [19], and ProbTS [95] compare statistical, deep, and (in some cases) foundation models, but differ in task taxonomies, splits, and leakage controls. Standardized metrics such as MASE [100] and CRPS [101] enable cross-dataset aggregation of point and probabilistic performance, yet consistent pretraining/evaluation protocols and leakage-free large-scale corpora remain key needs for fair TSFM assessment.

The consequence of these myriad issues is a research environment where it is difficult to distinguish genuine methodological advances from circumstantial performance on a narrow, and potentially contaminated, set of tasks. This is particularly damaging for the development of TSFMs. The significant computational and financial resources required to pre-train these models demand a rigorous, fair, and comprehensive evaluation framework to justify their development and guide future research [39]. The current state of affairs falls short of this standard. Indeed, studies have shown that existing TSFMs, often pre-trained on general-purpose academic datasets, can struggle to generalize to the unique and challenging characteristics of specialized domains like observability data [102].

The field has thus reached an inflection point. Progress is no longer limited by our ability to design novel model architectures, but by our inability to reliably and fairly measure their performance. Recognizing this crisis, recent efforts have focused on creating the next generation of evaluation infrastructure. The development of large-scale, standardized benchmarks such as GIFT-Eval and the domain-specific Benchmark of Observability Metrics (BOOM) represent a direct and necessary response [35]. These

initiatives introduce carefully curated and decontaminated pre-training and evaluation sets, standardized protocols, and data that reflects the complexity of real-world applications. They treat the benchmark not as a mere dataset, but as a carefully designed scientific instrument [28]. A clear and urgent research gap has thus emerged: the critical need for a new, large-scale, and meticulously curated public benchmark to serve as a gold standard for evaluating the next generation of time-series models. Such a contribution is not merely a prerequisite for future research; it provides essential infrastructure required to move the field from an era of fragmented claims to one of robust, reproducible, and generalizable progress [6], a foundational scientific contribution in its own right.

Contemporary time-series data seldom conform to the idealized assumptions of stationarity and linearity that underpin classical models. Instead, real-world data streams are characterized by a confluence of complex, interacting properties that present formidable modeling challenges [37].

- **Non-Linearity:** Perhaps the most fundamental challenge is the prevalence of non-linear relationships. Economic systems, biological processes, and energy grids are governed by complex feedback loops and interactions that cannot be adequately captured by linear models [35]. Traditional methods like Autoregressive Integrated Moving Average (ARIMA) are, by their construction, limited in their ability to model such non-linear dynamics, which is a primary reason for their performance ceiling on complex, real-world problems [37].
- **Multi-Regime Behavior:** Many time series exhibit structural breaks or distinct operational regimes, where the underlying data-generating process changes over time [25]. Examples include the shift between bull and bear markets in financial data or the different performance characteristics of an industrial machine under varying loads and environmental conditions. A single, global model often fails to capture this complex inner structure, leading to significant predictive errors when the system transitions between regimes [103].
- **Intermittency:** As noted previously, intermittent demand patterns are characterized by a high proportion of zero-valued observations, with non-zero demands occurring sporadically. This dual source of randomness, in both the timing and the magnitude of events, violates the continuity and regular sampling assumptions inherent in many classical smoothing and regression-based techniques [25].
- **Heightened Volatility and Novel Data Sources:** The modern data ecosystem is characterized by the emergence of new data sources that introduce unprecedented levels of volatility and complexity [25]. The integration of renewable energy sources into power grids is a prime example, creating load patterns with high-frequency noise and non-stationary behavior that challenge traditional forecasting approaches. A parallel development is the explosion of "observability data" generated by large-scale distributed software and cloud computing systems. This data, which includes metrics on CPU load, network latency, and application error rates, is often characterized by extreme non-stationarity, high dimensionality (thousands of correlated variables), heavy-tailed distributions, and sparsity, posing a unique and difficult set of modeling challenges [25].

F.1 An Arms Race of Methodological Innovation

The progression of forecasting methodologies can be understood as a direct response to this escalating data complexity. Each new paradigm has sought to overcome the limitations of its predecessors, leading to the current diverse and powerful toolkits available to researchers and practitioners [25].

- **The Classical Foundation:** The field was built upon a foundation of statistical methods developed primarily in the mid-20th century. Models such as ARIMA and its variants [28], Holt-Winters

Exponential Smoothing [25], and the Theta method [39] became the workhorses of the discipline. These models excel at capturing and extrapolating clear patterns of trend and seasonality from univariate time series. Their enduring appeal lies in their statistical rigor, interpretability, and computational efficiency. However, their reliance on strong assumptions about the underlying data-generating process, particularly linearity and stationarity, fundamentally limits their applicability to the more complex data common today [28].

- **The Machine Learning Advance:** The rise of machine learning in the late 20th and early 21st centuries provided a new set of tools capable of addressing the challenge of non-linearity. Non-parametric models like Support Vector Regression (SVR) [38] offered a principled approach, grounded in statistical learning theory, to model non-linear relationships in high-dimensional spaces via the "kernel trick" [35]. Concurrently, ensemble methods, particularly those based on decision trees like Random Forest and Gradient Boosting (e.g., XGBoost), proved to be exceptionally powerful and robust [37]. By combining the predictions of many weak learners, these models can capture complex, non-linear interactions and have consistently demonstrated state-of-the-art performance in a wide array of forecasting competitions and applications.
- **The Deep Learning Revolution:** While machine learning ensembles excelled at capturing complex feature interactions, they were not explicitly designed to model the long-range temporal dependencies inherent in sequential data. This limitation was addressed by the deep learning revolution. Recurrent Neural Networks (RNNs), and more specifically architectures like Long Short-Term Memory (LSTM) networks, were developed with internal memory mechanisms (gates) that allow them to capture and retain information over long sequences [35]. Empirical studies have shown that on complex financial and economic data, LSTMs can significantly outperform classical models like ARIMA by better modeling non-linear temporal dynamics [25]. Following the success of LSTMs, Transformer-based architectures, with their self-attention mechanism, have emerged as the next frontier, offering a powerful alternative for capturing dependencies across time without the sequential processing limitations of RNNs [35].

This co-evolution of data challenges and modeling paradigms, summarized in Table 14, illustrates a clear trajectory towards models of increasing complexity and representational power.

However, this progression is not a simple linear march where newer, more complex models invariably render older ones obsolete. Empirical evidence reveals a more nuanced reality, one that aligns with the well-known "No Free Lunch" theorem in machine learning. While deep learning models like LSTMs have been shown to decisively outperform ARIMA on certain complex datasets, recent large-scale studies have also found that in zero-shot or limited-supervision settings, simpler statistical methods often outperform sophisticated deep learning models [35]. Furthermore, in production environments like large-scale observability systems, classical models remain prevalent due to the operational infeasibility of training and maintaining millions of distinct, complex neural network models [25]. This apparent contradiction is not a flaw in the research, but rather a reflection of a fundamental truth: the performance of any given forecasting model is highly contingent on the specific characteristics of the data, the length of the forecast horizon, the availability of computational resources, and the degree of supervision. This recognition implies that the central problem in the field is not merely the invention of more powerful algorithms, but the development of a deeper, more systematic understanding of the complex performance landscape that governs the interaction between data characteristics and model architectures [35].

Table 14. The Co-evolution of Time-Series Challenges and Modeling Paradigms [25, 28, 35, 37, 39].

Era	Primary Data Challenge(s)	Dominant Model Paradigm	Key Models	Inherent Limitations
Statistical	Trends, Seasonality, Stationarity	Time-Domain Statistical Models	ARIMA, Holt-Winters, Theta	Struggle with non-linearity and complex dependencies
Machine Learning	Non-Linearity, Complex Interactions	Non-parametric	Ensemble Models (SVR, Random Forest, XGBoost)	Limited handling of long-range temporal dependencies
Deep Learning	Long-Range Dependencies, Sequential Patterns	Recurrent, Attention-based Networks	LSTMs, Transformers	Data-hungry, computationally intensive, task-specific
Foundation Models	Data Heterogeneity, Scale, Task Generalization	Large Pre-trained Models	MOMENT, TOTO, Chronos	Reliance on massive, curated datasets; evaluation bottleneck

F.2 The New Frontier: Pre-trained Foundation Models for Time Series

In response to the challenges of data heterogeneity and the high cost of developing task-specific models, the field is currently undergoing another paradigm shift, mirroring recent transformations in natural language processing and computer vision: the move towards large, pre-trained Time-Series Foundation Models (TSFMs). This new frontier aims to leverage the power of large-scale, self-supervised learning to create general-purpose models that can be adapted to a wide range of downstream tasks with minimal fine-tuning [35].

The core premise of the foundation model paradigm is to pre-train a single, high-capacity model (typically a Transformer) on a massive and diverse corpus of unlabeled data. This process allows the model to learn a rich, generalizable representation of temporal patterns. Subsequently, this pre-trained model can serve as a powerful building block for various downstream applications, including long- and short-horizon forecasting, time-series classification, anomaly detection, and missing value imputation. Models such as MOMENT, Chronos, and TOTO are at the vanguard of this movement. They are designed to be effective "out-of-the-box," providing strong zero-shot or few-shot performance without the need for extensive task-specific training [6] [13]. This approach holds particular promise for domains like observability, where the sheer scale and diversity of time series—often numbering in the millions or billions—make the traditional approach of training one model per series operationally intractable [37].

The primary enabler of this paradigm, and simultaneously its greatest bottleneck, is the availability of data. The success of foundation models in other domains was built on the existence of vast, cohesive, and publicly accessible datasets like The Pile for text [104] and ImageNet for vision [105]. The time-series domain, by contrast, has historically been characterized by a fragmented landscape of smaller, scattered, and task-specific public datasets [25]. This data scarcity has been a major impediment to large-scale pre-training. To overcome this, pioneering research efforts have begun the monumental task of data curation.

The creators of MOMENT compiled *The Time Series Pile*, a large collection of public repositories, while the TOTO model was pre-trained on a corpus containing a mixture of public, synthetic, and large-scale proprietary observability data, resulting in a dataset 4 to 10 times larger than those used for other leading TSFMs [13].

This focus on data curation signals a significant maturation of the field. In earlier eras, the primary axis of innovation was model architecture—for example, the design of the gating mechanisms in an LSTM or a novel attention variant in a Transformer [35]. The advent of the TSFM paradigm, however, shifts the research bottleneck. While architectural innovation remains important, the most critical and scientifically challenging work is now increasingly centered on the curation of massive, diverse, and clean datasets, and on the development of robust frameworks for evaluating the models trained on them. The value proposition of a new TSFM is now as much about the data it was trained on and the benchmark it was tested against as it is about its internal architecture. This implies that the most impactful contributions in this new era may not be designing a marginally better model, but rather curating the data and creating the evaluation infrastructure that enables the field to advance [28].

G NeurIPS Paper Checklist

1. **Claims**

   Answer: [Yes]

   Justification: The abstract and introduction clearly state the four problems motivating TempusBench (outdated/contaminated datasets, narrow benchmark task types, inconsistent hyperparameter tuning, and lack of visualization tools) and precisely map each to a corresponding contribution (70 new datasets, novel task taxonomy, standardized tuning protocol, TensorBoard-based visualization interface). The experimental results in Tables 3–5 directly support the scope of these claims.

2. **Limitations**

   Answer: [Yes]

   Justification: The paper discusses limitations including that TempusBench currently evaluates standard forecasting task types (horizon, variate type, context length, frequency) and does not yet cover all possible task dimensions. The benchmark is scoped to open-source TSFMs, and closed-source models are not included. Limitations of aggregation metrics (win rate, skill score) in the presence of metric correlation are acknowledged.

3. **Theory assumptions and proofs**

   Answer: [N/A]

   Justification: TempusBench is an empirical benchmarking framework and does not include novel theoretical results, theorems, or proofs.

4. **Experimental result reproducibility**

   Answer: [Yes]

   Justification: Full code and datasets are publicly available at `https://github.com/Smlcrm/TempusBench`. The paper describes the rolling-window evaluation protocol, the hyperparameter tuning procedure, and the 53 model implementations in sufficient detail to reproduce all reported results. A live interactive leaderboard is hosted at `https://smlcrm.com/tempusbench`.

5. **Open access to data and code**

   Answer: [Yes]

Justification: All 70 new datasets and all 53 model implementations are released open-source on GitHub (https://github.com/Smlcrm/TempusBench). The live leaderboard is hosted on Hugging Face. Instructions for running the full evaluation pipeline are included in the repository.

6. **Experimental setting/details**

   Answer: [Yes]

   Justification: The paper specifies the rolling-window evaluation design, the standardized hyperparameter tuning protocol applied uniformly across all model classes, the metrics used (MASE, MAE, RMSE, CRPS, WIS, QS, win rate, skill score), and the benchmark task taxonomy (Tables 2 and 9). Model families and dataset properties are documented in the appendices.

7. **Experiment statistical significance**

   Answer: [Yes]

   Justification: Results are reported as average win rates across metrics and tasks (Tables 3–5), which are inherently aggregated over multiple datasets and rolling windows, providing robustness against single-run variance. The win rate formulation provides a nonparametric summary of pairwise performance differences across the full benchmark suite.

8. **Experiments compute resources**

   Answer: [Yes]

   Justification: The paper documents the compute requirements for running the evaluation pipeline across 53 models on 70 datasets under rolling-window evaluation, including the hardware used and approximate per-model runtime. This information is sufficient for others to estimate reproduction costs.

9. **Code of ethics**

   Answer: [Yes]

   Justification: The research involves no human subjects, no sensitive data, and no dual-use risks. All datasets are either newly collected or derived from publicly available time-series sources. The work conforms with the NeurIPS Code of Ethics.

10. **Broader impacts**

    Answer: [Yes]

    Justification: The paper discusses positive societal impacts (improving rigor and reproducibility in TSFM evaluation, democratizing access to benchmarking tools) and acknowledges potential negative impacts, such as the possibility that a public leaderboard could be gamed or that benchmark-specific overfitting could develop over time as models are tuned against TempusBench tasks.

11. **Safeguards**

    Answer: [N/A]

    Justification: TempusBench is a benchmarking and evaluation framework. It does not release a generative model, image generator, or scraped dataset with meaningful misuse risk. No special safeguards beyond standard open-source licensing are required.

12. **Licenses for existing assets**

    Answer: [Yes]

    Justification: All existing datasets and model codebases used are properly cited with original papers, version information where available, and license terms. Models such as Chronos-2 (Apache 2.0),

TiRex, TabPFN, and others are attributed. Dataset licenses are documented in the supplemental material.

13. **New assets**

    Answer: [Yes]

    Justification: The 70 new datasets introduced by TempusBench are documented with metadata (domain, frequency, variate type, stationarity, seasonality, size class). The framework codebase is released under an open-source license with a README, environment specifications, and usage instructions. No human subject data requiring consent documentation is included.

14. **Crowdsourcing and research with human subjects**

    Answer: [N/A]

    Justification: The paper does not involve crowdsourcing or research with human subjects. All data is observational time-series data from public or newly compiled sources.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

    Answer: [N/A]

    Justification: No human subjects research is conducted. IRB approval is not applicable.

16. **Declaration of LLM usage**

    Answer: [N/A]

    Justification: LLMs are not a component of TempusBench's core methodology. Any use of LLMs was limited to writing and editing assistance, which does not require declaration under NeurIPS policy.